

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería



Desarrollo de sistema de posicionamiento preciso en interiores mediante sensores UWB para recorridos virtuales con realidad aumentada en la Universidad del Valle de Guatemala.

Trabajo de graduación en modalidad de Trabajo Profesional presentado por Gustavo Andres González Pineda para optar al grado académico de Licenciado en Ingeniería en Ciencias de la Computación y Tecnologías de la Información.

Guatemala, 2025

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería



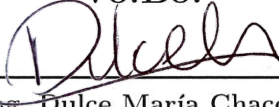
Desarrollo de sistema de posicionamiento preciso en interiores mediante sensores UWB para recorridos virtuales con realidad aumentada en la Universidad del Valle de Guatemala.

Trabajo de graduación en modalidad de Trabajo Profesional presentado por Gustavo Andres González Pineda para optar al grado académico de Licenciado en Ingeniería en Ciencias de la Computación y Tecnologías de la Información.

Guatemala, 2025

Vo.Bo.

Firma:


Ing. Dulce María Chacón

Firma:


Ing. Cecilia Castillo

Fecha de aprobación: Guatemala, 18 de noviembre de 2025.

Índice

1. Lista de cuadros y figuras	I
2. Abstract	II
3. Resumen	III
4. Introducción	1
5. Justificación	2
6. Objetivos	3
6.1. Objetivo general	3
6.2. Objetivos específicos	3
7. Marco Teórico	4
7.1. Localización en interiores	4
7.2. Tecnología Ultra-Wideband (UWB)	4
7.3. Trilateración	4
7.4. Filtro de Kalman	5
7.5. Acelerómetro	5
7.6. Unity y Realidad Aumentada	5
7.7. Plugins Nativos para Unity (iOS)	6
8. Metodología	7
8.1. Introducción	7
8.2. Tipo de investigación	7
8.3. Metodología estructurada por objetivos	7
8.3.1. Objetivo 1: Diseñar e implementar un algoritmo de trilateración para estimar la posición 2D del usuario	7
8.3.2. Herramientas y conocimientos aplicados:	7
8.3.3. Objetivo 2: Integrar el SDK nativo de Estimote con Unity mediante un plugin de comunicación en tiempo real	8

8.3.4. Herramientas y conocimientos aplicados:	8
8.3.5. Objetivo 3: Aplicar filtrado con técnicas como el filtro de Kalman para reducir el jitter	8
8.3.6. Herramientas y conocimientos aplicados:	8
8.3.7. Objetivo 4: Documentar el desarrollo del plugin y su integración	8
8.3.8. Herramientas y conocimientos aplicados:	9
8.3.9. Objetivo 5: Evaluar cualitativamente la precisión y suavidad del sistema de localización	9
8.3.10. Herramientas y conocimientos aplicados:	9
8.4. Recolección de datos	9
8.5. Análisis de datos	10
8.6. Consideraciones éticas	10
9. Resultados y Discusión	11
9.1. Objetivo 1: Algoritmo de trilateración	11
9.1.1. Fase1: Diseño del algoritmo	11
9.1.2. Fase 2: Desarrollo del algoritmo	12
9.1.3. Fase 3: Validación funcional	15
9.2. Objetivo 2: Integrar el SDK nativo de Estimote con Unity mediante un plugin de comunicación en tiempo real	17
9.3. Objetivo 3: Aplicación del filtro de Kalman para reducir el jitter y fusionar datos sensoriales	20
9.4. Objetivo 4: Documentación del desarrollo e integración del plugin	26
9.5. Objetivo 5: Evaluar cualitativamente la precisión y suavidad del sistema de localización	27
9.6. Conclusiones	31
10.Recomendaciones	32
11.Bibliografía	33
12.Anexos	34
12.1. Implementación del método de multilateración	34
12.2. Implementación del método trilaterate 3D	34
12.3. Implementación completa del algoritmo de trilateración en Swift	35

12.4. Glosario	38
--------------------------	----

1. Lista de cuadros y figuras

Índice de figuras

1.	Proyección del vector $\mathbf{b}$ sobre el espacio columna de A , base conceptual de las ecuaciones normales. .	14
2.	Gráfica de la magnitud del vector de posición a lo largo del tiempo, mostrando el jitter observado durante la trilateración 2D.	16
3.	Prueba de concepto con enfoque inicial basada en un solo sensor UWB.	18
4.	Prueba de concepto mediante escena de Unity, en azul el objeto tridimensional representando el dispositivo, en verde la ruta que debe seguir el objeto, en rojo la posición de destino.	19
5.	Crecimiento de la velocidad al integrar los datos del acelerómetro para estimar la posición.	20
6.	Distribución del acelerómetro en reposo registrada mediante <i>CoreMotion</i> . Los tres ejes presentan una varianza mínima, consistente con un estado de inmovilidad del dispositivo.	21
7.	Distribución de la posición estimada mediante trilateración en reposo. Se observan los tres ejes superpuestos.	22
8.	Comparación de la trayectoria diagonal del usuario en movimiento. La trayectoria filtrada muestra reducción de jitter y mayor suavidad respecto a la señal sin filtrar.	23
9.	Magnitudes de posición filtradas con Kalman.	24
10.	Arquitectura final del sistema tras la integración del filtro de Kalman.	25
11.	Pruebas domésticas con el sistema completamente integrado. Las trayectorias muestran un movimiento continuo y estable, confirmando la correcta sincronización entre el algoritmo de trilateración, el filtro de Kalman y la aplicación de realidad aumentada.	27
12.	Pruebas en el entorno universitario mostrando la trayectoria del usuario (izquierda) y la visualización en realidad aumentada (derecha). Ambas perspectivas confirman la coherencia entre la posición estimada y la dirección mostrada al usuario.	29
13.	Log de comunicación entre la aplicación y el SDK de Estimote, donde se origina el error de calendarización.	30

2. Abstract

This project aims to develop a precise location system for an augmented reality application that guides users around the campus of the Universidad del Valle de Guatemala. The system is based on Estimote Beacons with Ultra-Wideband (UWB) technology and is designed to integrate with Unity through a native plugin that enables real-time communication between mobile devices and location sensors.

Unlike previous solutions that relied on QR codes or sensor fusion techniques subject to cumulative errors, this proposal employs Kalman filter-assisted trilateration to more reliably estimate the user's position in 2D coordinates. The system will be able to transmit this data to Unity, where it will be used to render augmented reality elements that guide the user around the campus.

The project is part of a larger ongoing initiative and focuses on the technical implementation of the positioning module, laying the groundwork for future cross-platform extensions. This technological solution significantly contributes to improving the on-campus wayfinding experience and represents an example of the practical use of emerging technologies in localization, signal filtering, and interactive mobile application development.

3. Resumen

Este proyecto tiene como objetivo desarrollar un sistema de localización precisa para una aplicación de realidad aumentada que guía a los usuarios dentro del campus de la Universidad del Valle de Guatemala. El sistema se basa en sensores Estimote Beacons con tecnología Ultra-Wideband (UWB) y está diseñado para integrarse con Unity mediante un plugin nativo que permite la comunicación en tiempo real entre los dispositivos móviles y los sensores de localización.

A diferencia de soluciones anteriores que dependían de códigos QR o técnicas de sensor fusion sujetas a errores acumulativos, esta propuesta emplea trilateración asistida por filtros de Kalman para estimar con mayor estabilidad la posición del usuario en coordenadas 2D. El sistema será capaz de transmitir estos datos a Unity, donde se utilizan para renderizar elementos de realidad aumentada que guían al usuario por el campus.

El proyecto forma parte de una iniciativa mayor en curso y se enfoca en la implementación técnica del módulo de posicionamiento, sentando las bases para futuras extensiones multiplataforma. Esta solución tecnológica contribuye significativamente a mejorar la experiencia de orientación dentro del campus y representa un ejemplo del uso práctico de tecnologías emergentes en localización, filtrado de señales y desarrollo de aplicaciones móviles interactivas.

4. Introducción

La orientación dentro de un campus universitario puede representar un desafío para estudiantes nuevos, visitantes y personas con discapacidad. Con el avance de la tecnología, la realidad aumentada (AR) se ha convertido en una herramienta innovadora para mejorar la experiencia de navegación en espacios físicos. En este contexto, el presente proyecto busca desarrollar una aplicación de realidad aumentada para recorridos dentro de la Universidad del Valle de Guatemala (UVG), proporcionando una guía interactiva e inclusiva que facilite el desplazamiento desde un punto A hasta un punto B dentro del campus.

Este proyecto se basa en el trabajo realizado en años anteriores, donde se implementó una versión preliminar de la navegación con AR, pero con limitaciones significativas. Una de las principales dificultades fue la dependencia de códigos QR para la ubicación, lo que restringía la precisión y fluidez del recorrido. Además, la implementación del algoritmo de pathfinding presentaba problemas en la generación de rutas óptimas, afectando la experiencia del usuario.

Para mejorar estos aspectos, la universidad ha invertido en sensores Estimote Beacons con tecnología Ultra-Wideband (UWB), los cuales permitirán una localización más precisa dentro del campus. La labor de este trabajo se centrará en el mapeo del campus para determinar la ubicación óptima de estos sensores, su configuración e integración con la aplicación y la optimización del algoritmo de pathfinding para generar rutas más accesibles y eficientes.

El objetivo es mejorar la precisión de la navegación y evitar rutas poco prácticas o inaccesibles para ciertos usuarios. Esto contribuirá a una experiencia más fluida y efectiva al desplazarse dentro del campus, garantizando que la aplicación proporcione indicaciones precisas y usables en diferentes escenarios. A través de este trabajo, se espera ofrecer una solución innovadora y funcional que aproveche las capacidades de la realidad aumentada y la localización UWB para facilitar la movilidad dentro de la UVG, mejorando la orientación y accesibilidad dentro del campus universitario.

5. Justificación

Actualmente, el proceso de orientación para estudiantes nuevos, visitantes y padres de familia dentro del campus universitario de la Universidad del Valle de Guatemala presenta diversas limitaciones. Tradicionalmente, el recorrido por las instalaciones es guiado por estudiantes o personal universitario, pero este se realiza una única vez —si es que se llega a realizar— lo cual suele ser insuficiente para que los usuarios recuerden rutas, espacios o servicios disponibles. Como consecuencia, durante los primeros semestres, muchos estudiantes enfrentan dificultades para ubicarse, llegando a perderse o desaprovechar recursos valiosos del campus.

Además de la desorientación inicial, existen barreras de accesibilidad que afectan a personas con movilidad reducida o con discapacidades visuales, quienes podrían beneficiarse significativamente de un sistema que les permita planificar sus trayectos de forma precisa y adaptada a sus necesidades. Una solución tecnológica puede contribuir a solventar ambos problemas, mejorando tanto la orientación como la inclusión dentro del entorno universitario.

En este contexto, el desarrollo de una aplicación de realidad aumentada con capacidades de localización y navegación precisa representa una oportunidad para aplicar conocimientos técnicos adquiridos a lo largo de la carrera de Ingeniería en Ciencias de la Computación y Tecnologías de la Información. Este proyecto integra áreas clave como programación orientada a objetos, interacción humano-computadora, gráficos por computadora, desarrollo móvil y algoritmos de búsqueda de caminos (pathfinding). Asimismo, se utilizarán sensores Estimote Beacons con tecnología Ultra-Wideband (UWB), cuya precisión y facilidad de conexión brindan una ventaja considerable en comparación con tecnologías previamente utilizadas, como los códigos QR.

Este trabajo, al ser parte de un megaproyecto en curso, busca fortalecer y mejorar la versión previa de la aplicación desarrollada por generaciones anteriores, proponiendo avances significativos en la precisión de la localización y en la experiencia del usuario. La solución no solo resolverá problemas existentes, sino que también funcionará como base tecnológica para futuras generaciones de estudiantes que podrán continuar desarrollando y afinando la herramienta.

Finalmente, este proyecto aporta a la universidad al fortalecer su imagen como institución innovadora, capaz de generar soluciones tecnológicas útiles y vanguardistas dentro de su propio entorno. Comunica el potencial de sus estudiantes para abordar problemas reales mediante el uso creativo y riguroso del conocimiento adquirido, posicionando a la UVG como una universidad comprometida con la tecnología, la accesibilidad y la experiencia estudiantil.

6. Objetivos

6.1. Objetivo general

Desarrollar e implementar un plugin de localización en tiempo real mediante sensores Estimote con tecnología Ultra-Wideband (UWB), integrado a Unity, como parte de una aplicación de realidad aumentada que facilite los recorridos virtuales dentro del Centro de Innovación y Tecnología (CIT) del campus de la Universidad del Valle de Guatemala.

6.2. Objetivos específicos

- 6.2.1 Diseñar e implementar un algoritmo de trilateración para estimar la posición bidimensional del usuario dentro del campus universitario, mediante el procesamiento de las distancias medidas hacia múltiples sensores Estimote UWB distribuidos en el entorno.
- 6.2.2 Integrar el SDK nativo de Estimote para iOS con Unity para permitir la obtención y transferencia en tiempo real de datos de localización hacia la aplicación de realidad aumentada, mediante el desarrollo de un plugin nativo en Swift que sirva de puente entre ambas plataformas.
- 6.2.3 Aplicar técnicas de filtrado de señales para mejorar la estabilidad del sistema de localización y reducir el efecto de ruido o jitter en las mediciones, mediante la combinación de los datos obtenidos por trilateración y los registros del acelerómetro del dispositivo.
- 6.2.4 Documentar el desarrollo del sistema de localización para asegurar su mantenibilidad, facilitar su futura extensión a dispositivos Android y permitir su continuidad, mediante la elaboración de documentación técnica clara sobre la arquitectura del plugin, las decisiones de diseño y los procesos de integración.
- 6.2.5 Evaluar cualitativamente la precisión y suavidad del sistema de localización para validar su funcionamiento en condiciones reales y su aporte a la experiencia de usuario dentro de la aplicación, mediante la observación directa del comportamiento del sistema y el análisis de las distribuciones de datos de posición obtenidas durante pruebas manuales.

7. Marco Teórico

7.1. Localización en interiores

La localización en interiores, también conocida como Indoor Positioning, es un área de estudio dentro de la computación ubicua que busca determinar la posición de personas u objetos dentro de espacios cerrados, donde las señales satelitales como el GPS pierden cobertura o presentan alta inexactitud. A diferencia de los sistemas de localización en exteriores, la implementación de soluciones en interiores implica múltiples retos técnicos, como la interferencia de estructuras físicas (paredes, techos, mobiliario), el comportamiento impredecible de las señales de radiofrecuencia por efectos como la reflexión, refracción y difracción, así como la variabilidad en el entorno.

Para suplir estas limitaciones, se han explorado diferentes tecnologías de localización alternativas, entre ellas Wi-Fi, Bluetooth Low Energy (BLE), RFID, visión por computadora y Ultra-Wideband (UWB). Cada tecnología ofrece un compromiso distinto entre precisión, consumo energético, costo y complejidad de implementación. En entornos que requieren alta precisión —como hospitales, bodegas o, en este caso, instalaciones universitarias complejas—, el UWB se destaca por ofrecer ventajas clave frente a sus competidores, sobre todo en términos de exactitud, latencia y robustez frente al ruido ambiental.

7.2. Tecnología Ultra-Wideband (UWB)

La tecnología Ultra-Wideband (UWB) se basa en el uso de pulsos de muy corta duración que se propagan en un espectro extremadamente amplio (mayor a 500 MHz). Esta característica permite transmitir datos con alta precisión temporal, lo que a su vez facilita la medición del tiempo exacto que tarda una señal en desplazarse entre dos dispositivos. A través de estos tiempos de vuelo (Time of Flight, ToF), un receptor puede estimar con mucha precisión la distancia a un emisor, sin depender de la intensidad de la señal (RSSI), como ocurre en tecnologías menos precisas como BLE o Wi-Fi.

En aplicaciones de localización, los dispositivos UWB se colocan en el entorno como anclas o beacons, que transmiten pulsos periódicamente. Un dispositivo móvil equipado con un receptor UWB puede calcular su distancia a múltiples beacons y usar dicha información para estimar su posición. Esta técnica, que combina alta precisión (en el orden de decímetros o incluso centímetros) con baja latencia, es ideal para aplicaciones donde se requiere posicionamiento en tiempo real con gran fidelidad espacial, como la navegación asistida con realidad aumentada. En este proyecto se utilizan sensores Estimote UWB Beacons, diseñados específicamente para tareas de localización en interiores, con soporte en plataformas móviles como iOS.

7.3. Trilateración

La trilateración es un método geométrico fundamental en sistemas de localización que permite calcular la posición de un punto desconocido en el espacio utilizando las distancias conocidas a tres o más puntos de referencia cuyas ubicaciones son fijas. En el caso de localización en dos dimensiones, si se conocen las coordenadas de tres sensores (beacons) y las distancias del dispositivo móvil a cada uno de ellos, es posible determinar su posición mediante la intersección de los tres círculos generados por esas distancias. En la práctica, debido a errores de medición, estas circunferencias rara vez se intersectan en un solo punto, por lo que se recurre a métodos de optimización y regresión para encontrar el punto más probable.

Cuando se disponen más de tres sensores, se puede aplicar trilateración redundante, técnica que permite mejorar la estabilidad de la estimación al reducir el impacto de errores individuales en las mediciones. La trilateración es la base matemática de muchos sistemas de localización, incluidos GPS, y en este proyecto, se adapta al contexto del

campus universitario, utilizando coordenadas cartesianas en un plano 2D para determinar la ubicación del usuario en tiempo real.

7.4. Filtro de Kalman

El filtro de Kalman es un algoritmo recursivo de estimación desarrollado por Rudolf E. Kálmán en la década de 1960, utilizado para predecir el estado de un sistema dinámico y corregir esa predicción a partir de mediciones ruidosas. Su aplicación ha sido clave en campos como la navegación inercial, la robótica, el seguimiento de objetos y los sistemas de control, debido a su capacidad para integrar múltiples fuentes de información en presencia de incertidumbre.

En el contexto de este proyecto, el filtro de Kalman se emplea para suavizar la trayectoria estimada del usuario dentro del campus, combinando las posiciones calculadas mediante trilateración con los datos del acelerómetro del dispositivo móvil. Esta fusión sensorial permite reducir el jitter, es decir, las pequeñas oscilaciones aleatorias en la posición del usuario, y mejorar la estabilidad de la trayectoria percibida. Aunque el sistema no busca una precisión milimétrica absoluta, el objetivo es proporcionar una experiencia de navegación fluida, donde la posición calculada varíe de forma coherente con el movimiento del usuario.

7.5. Acelerómetro

El acelerómetro es un sensor electrónico capaz de medir la aceleración de un cuerpo en los tres ejes del espacio (x , y , z), tanto por efecto del movimiento como por la fuerza gravitacional. En los dispositivos móviles modernos, los acelerómetros permiten funciones como la detección de orientación, el conteo de pasos, y la navegación sin GPS (dead reckoning).

En este proyecto, el acelerómetro se utiliza como complemento a la trilateración UWB para mejorar la estimación de la posición del usuario. Aunque los acelerómetros presentan el inconveniente de la deriva acumulativa (pequeños errores que se amplifican con el tiempo cuando se integra la aceleración), su uso combinado con los datos de distancia de los sensores UWB, mediante el filtro de Kalman, permite detectar transiciones suaves en el movimiento del usuario, anticipar su dirección y estabilizar la trayectoria. De esta manera, el sistema puede ofrecer una experiencia más coherente y natural en la navegación con realidad aumentada.

7.6. Unity y Realidad Aumentada

Unity es un motor de desarrollo multiplataforma ampliamente utilizado en la industria de los videojuegos, simulaciones y aplicaciones interactivas. Su compatibilidad con dispositivos móviles y su integración con librerías de realidad aumentada como ARKit (iOS) y ARCore (Android), lo convierten en una herramienta poderosa para el desarrollo de experiencias inmersivas en tiempo real.

En este proyecto, Unity se emplea como el entorno de ejecución principal para la aplicación de recorridos virtuales con realidad aumentada. A través de la cámara del dispositivo móvil, el usuario puede observar su entorno mientras el sistema le superpone elementos gráficos —como flechas tridimensionales— que le indican hacia dónde debe avanzar para completar un recorrido. La interacción entre estos elementos visuales y el sistema de localización precisa permite convertir el espacio físico del campus en una experiencia guiada interactiva, intuitiva y accesible.

7.7. Plugins Nativos para Unity (iOS)

Debido a que Unity no ofrece acceso directo a todas las funcionalidades del sistema operativo, como sensores especializados o bibliotecas nativas, es común extender su funcionalidad mediante plugins nativos. Un plugin nativo permite conectar el código de Unity con código escrito en lenguajes como Swift (para iOS) o Kotlin (para Android), habilitando el uso de SDKs que solo están disponibles de forma nativa.

En el caso de este proyecto, se desarrolla un plugin en Swift que se encarga de establecer la comunicación con los sensores Estimote UWB, procesar las mediciones de distancia, acceder a los datos del acelerómetro mediante el framework CoreMotion, realizar los cálculos de trilateración y aplicar el filtro de Kalman. Este plugin se comunica con Unity a través de una interfaz de funciones expuestas, enviando las coordenadas calculadas para ser usadas por el motor gráfico en la navegación. Esta arquitectura modular garantiza que el sistema pueda mantenerse y extenderse fácilmente, incluyendo la posibilidad futura de implementar un plugin análogo para Android.

8. Metodología

8.1. Introducción

Para alcanzar los objetivos planteados en este proyecto, se ha diseñado una metodología de carácter **experimental**, centrada en la integración de sensores UWB, el desarrollo de plugins nativos para iOS y su comunicación con el motor gráfico Unity. El enfoque se basa en el diseño, implementación, prueba y ajuste de algoritmos y módulos de software dentro de un entorno controlado y real, específicamente el edificio del Centro de Innovación y Tecnología de la Universidad del Valle de Guatemala.

8.2. Tipo de investigación

La investigación es de tipo **experimental aplicada**, ya que se basa en la validación empírica de una solución tecnológica diseñada para resolver un problema específico del entorno universitario. A través de pruebas funcionales, análisis de datos y evaluación cualitativa, se busca comprobar la viabilidad y efectividad de un sistema de localización en tiempo real, integrable con una aplicación de realidad aumentada.

8.3. Metodología estructurada por objetivos

8.3.1. Objetivo 1: Diseñar e implementar un algoritmo de trilateración para estimar la posición 2D del usuario

- **Fase 1: Diseño del algoritmo**

Se definirá un modelo matemático basado en trilateración 2D utilizando coordenadas cartesianas. El algoritmo tomará como entrada las distancias a múltiples sensores UWB y devolverá una estimación de posición en el plano.

- **Fase 2: Desarrollo del algoritmo**

El algoritmo será implementado en Swift como parte del plugin nativo para iOS. Se probará inicialmente en entornos controlados utilizando distancias simuladas y luego con datos reales de los sensores Estimote.

- **Fase 3: Validación funcional**

Se realizarán pruebas en campo para verificar la estabilidad del algoritmo, observando la coherencia espacial de las posiciones calculadas frente al movimiento del usuario.

8.3.2. Herramientas y conocimientos aplicados:

Para el desarrollo de este objetivo se aplican

conocimientos de geometría analítica para modelar matemáticamente la trilateración en dos dimensiones, utilizando coordenadas cartesianas. Asimismo, se emplea programación orientada a objetos en el lenguaje Swift para implementar el algoritmo dentro de un entorno nativo para iOS. Se recurre también a principios básicos de teoría de localización y análisis espacial para interpretar las relaciones entre sensores y usuario, y garantizar que los cálculos de posición tengan coherencia estructural en un espacio real.

8.3.3. Objetivo 2: Integrar el SDK nativo de Estimote con Unity mediante un plugin de comunicación en tiempo real

- **Fase 1: Conexión con sensores UWB**

Se desarrollará un módulo nativo en Swift capaz de escanear, conectar y recibir datos de los sensores Estimote UWB, utilizando el SDK oficial proporcionado por el fabricante.

- **Fase 2: Desarrollo del plugin para Unity**

Se diseñará un plugin nativo que actúe como puente entre el código Swift y el entorno Unity, permitiendo transmitir datos de posicionamiento en tiempo real.

- **Fase 3: Validación de la comunicación**

Se probará la integración verificando la correcta recepción y actualización de coordenadas dentro de Unity, así como el rendimiento del sistema en condiciones reales de uso.

8.3.4. Herramientas y conocimientos aplicados:

Este objetivo requiere la aplicación de conocimientos en desarrollo móvil nativo, específicamente para iOS, a través del uso de Swift y del SDK oficial de Estimote para sensores UWB. Además, se requiere dominio de la integración de plugins nativos en Unity, lo cual implica comprender los mecanismos de interoperabilidad entre plataformas, la comunicación entre código nativo y el motor gráfico, y el paso eficiente de datos en tiempo real. También se aplican conceptos de diseño modular y arquitectura de software para estructurar el plugin de forma mantenible.

8.3.5. Objetivo 3: Aplicar filtrado con técnicas como el filtro de Kalman para reducir el jitter

- **Fase 1: Recolección de datos crudos**

Se recopilarán datos en tiempo real de la trilateración y del acelerómetro del dispositivo móvil mediante CoreMotion.

- **Fase 2: Implementación del filtro de Kalman**

Se desarrollará un módulo en Swift que aplique el filtro de Kalman a las posiciones estimadas, utilizando como entradas tanto las mediciones UWB como la aceleración registrada.

- **Fase 3: Análisis de estabilidad**

Se evaluará la efectividad del filtrado mediante visualización de trayectorias y análisis del ruido residual en las posiciones. Se priorizará la precisión (consistencia) sobre la exactitud absoluta.

8.3.6. Herramientas y conocimientos aplicados:

En este punto del proyecto se aplican fundamentos de estadística aplicada y procesamiento de señales para implementar técnicas de filtrado, con énfasis en el filtro de Kalman. También se integran conceptos relacionados con sensores inerciales como el acelerómetro y giroscopio, usando librerías como CoreMotion en iOS para complementar los datos de posición. Finalmente, se requiere la capacidad de programar estructuras de filtrado en tiempo real y analizar la relación entre precisión, estabilidad y ruido de la señal.

8.3.7. Objetivo 4: Documentar el desarrollo del plugin y su integración

- **Fase 1: Estructuración del código y módulos**

Se organizará el código del plugin de manera modular y reutilizable, con comentarios explicativos y convenciones de nomenclatura claras.

- **Fase 2: Elaboración de documentación técnica**

Se redactará documentación detallada que describa la arquitectura del sistema, el funcionamiento interno del plugin, los flujos de datos y los pasos para su integración y reutilización.

- **Fase 3: Publicación y entrega**

Toda la documentación será entregada junto al proyecto final y se dispondrá para futuras generaciones con el fin de facilitar la continuidad del megaproyecto.

8.3.8. Herramientas y conocimientos aplicados:

Para este objetivo se aplican buenas prácticas de desarrollo de software, como la escritura de código modular, comentado y con convenciones claras de nomenclatura. También se emplean técnicas de documentación técnica para describir el funcionamiento de cada módulo, la estructura del sistema, los flujos de datos y los pasos necesarios para su mantenimiento o extensión. Todo esto se enmarca dentro de un enfoque de diseño sostenible, que busca facilitar la transferencia del conocimiento a futuros desarrolladores del megaproyecto.

8.3.9. Objetivo 5: Evaluar cualitativamente la precisión y suavidad del sistema de localización

- **Fase 1: Pruebas de campo**

Se ejecutarán pruebas en zonas específicas del edificio para observar el comportamiento del sistema en movimiento y en reposo.

- **Fase 2: Recolección y análisis de datos**

Se analizarán trayectorias, jitter, y ruido en las posiciones generadas. Se observará la estabilidad visual del sistema para el usuario final.

- **Fase 3: Ajustes iterativos**

En caso de detectar irregularidades, se realizarán ajustes al algoritmo de trilateración o al filtro de Kalman hasta alcanzar una experiencia fluida.

8.3.10. Herramientas y conocimientos aplicados:

En esta etapa se aplican conocimientos de análisis de datos para interpretar trayectorias, identificar patrones de jitter y evaluar la continuidad del movimiento estimado por el sistema. Se utilizan herramientas de visualización para representar el comportamiento del sistema bajo diferentes condiciones de uso. También se recurre a métodos de prueba empírica y ajuste de parámetros para afinar el sistema con base en los resultados observados, priorizando una experiencia fluida por encima de una exactitud matemática absoluta.

8.4. Recolección de datos

Durante el desarrollo y validación del sistema se recopilarán los siguientes tipos de datos:

- Distancias entre el dispositivo móvil y sensores UWB.
- Coordenadas calculadas por el sistema de trilateración.
- Aceleración y orientación medidas por el acelerómetro y giroscopio del dispositivo.

- Registros de estabilidad (jitter) y continuidad de la trayectoria estimada.

Todas las pruebas se llevarán a cabo dentro del campus universitario en condiciones controladas, utilizando dispositivos iOS compatibles con UWB.

8.5. Análisis de datos

El análisis de datos se enfocará en:

- Cuantificar la estabilidad de las posiciones calculadas en reposo y en movimiento.
- Visualizar trayectorias para detectar ruido o errores de salto.
- Comparar el comportamiento del sistema con y sin filtrado aplicado.

Se utilizarán herramientas de análisis visual y gráficos generados mediante scripts personalizados en Swift o Python, según se requiera.

8.6. Consideraciones éticas

Este proyecto no involucra usuarios reales en esta etapa, ni recopila información sensible. Sin embargo, se contemplan las siguientes buenas prácticas para futuras fases o generaciones:

- Solicitar consentimiento informado si se realizan pruebas con participantes.
- Garantizar el anonimato de los datos de localización recolectados.
- Usar la información únicamente con fines académicos y de mejora del sistema.

Actualmente, todos los datos son técnicos y recopilados internamente por los desarrolladores en contextos controlados.

9. Resultados y Discusión

9.1. Objetivo 1: Algoritmo de trilateración

Para el cálculo de la posición del usuario se implementó un algoritmo de trilateración en dos dimensiones (x,y), dado que el propósito principal de la aplicación de realidad aumentada es determinar la ubicación del usuario sobre un plano, sin necesidad de considerar su altura exacta en el eje z. En un escenario tridimensional, la trilateración requiere al menos cuatro sensores UWB para resolver la posición completa del usuario. Sin embargo, al limitar el problema al plano bidimensional, fue posible emplear únicamente tres sensores, con la condición de manejar adecuadamente el eje z para evitar que su omisión se tradujera en un error adicional sobre los ejes x e y.

9.1.1. Fase1: Diseño del algoritmo

Derivación matemática del modelo

El modelo parte de la ecuación de distancia entre el usuario (x, y, z) y cada sensor i :

$$\sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} = d_i$$

Al elevar ambos lados al cuadrado y expandir los términos, se obtiene:

$$x^2 + y^2 + z^2 - 2x \cdot x_i - 2y \cdot y_i - 2z \cdot z_i + (x_i^2 + y_i^2 + z_i^2) = d_i^2$$

Para eliminar los términos cuadráticos comunes, se resta una ecuación de referencia (sensor 1) al resto, obteniendo un sistema lineal en las variables x , y y z :

$$2x(x_1 - x_i) + 2y(y_1 - y_i) + 2z(z_1 - z_i) = d_i^2 - d_1^2 - x_i^2 - y_i^2 - z_i^2 + x_1^2 + y_1^2 + z_1^2$$

Cada ecuación puede representarse en la forma general:

$$A_i x + B_i y + C_i z = D_i$$

donde los coeficientes son:

$$A_i = 2(x_1 - x_i), \quad B_i = 2(y_1 - y_i), \quad C_i = 2(z_1 - z_i), \quad D_i = d_i^2 - d_1^2 - x_i^2 - y_i^2 - z_i^2 + x_1^2 + y_1^2 + z_1^2$$

De esta manera, para n sensores se obtienen $n - 1$ ecuaciones lineales que pueden expresarse matricialmente como:

$$\begin{bmatrix} A_{1x} & A_{1y} & A_{1z} \\ A_{2x} & A_{2y} & A_{2z} \\ \vdots & \vdots & \vdots \\ A_{n-1x} & A_{n-1y} & A_{n-1z} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} A_{1d} \\ A_{2d} \\ \vdots \\ A_{n-1d} \end{bmatrix}$$

Dado que el interés del sistema se centra en la ubicación sobre el plano horizontal, se asumió que todos los sensores se encuentran a una misma altura ($z_i = z_1$), simplificando el sistema a dos dimensiones. Esta simplificación elimina el término asociado a z y reduce el problema a la intersección de tres circunferencias en el plano (x, y) .

El sistema puede producir dos posibles soluciones para la coordenada z cuando las circunferencias se intersectan, o bien soluciones imaginarias cuando no existe intersección exacta. Dado que la coordenada z no es relevante para la aplicación, se adoptó la siguiente estrategia:

- Si el discriminante era positivo, se resolvía completamente el sistema y se tomaba la solución con z positiva.
- Si el discriminante era negativo, se fijaba el valor de $z = 0$.

Este procedimiento permitió compensar parcialmente el efecto de la elevación en las mediciones, evitando que el error en la componente vertical se propagara hacia las coordenadas de interés. Para reforzar esta estrategia, se colocaron todos los sensores a una misma altura, de manera que la coordenada z asignada a cada beacon fuera arbitraria pero consistente en todo el sistema.

9.1.2. Fase 2: Desarrollo del algoritmo

Implementación del algoritmo en Swift

El algoritmo fue implementado en el lenguaje **Swift** como parte del plugin nativo de localización para iOS, como se muestra en el listado 3 de la sección de Anexos. La función principal, `do_trilateration`, recibe como entrada un conjunto dinámico de sensores activos, y selecciona automáticamente las tres anclas más recientes basándose en su marca de tiempo (*timestamp*). Esta estrategia de selección garantiza que las mediciones correspondan a señales coherentes temporalmente, evitando inconsistencias en el cálculo espacial.

Cada ancla contiene su posición (x_i, y_i, z_i) y la distancia medida d_i . Estos datos se utilizan para construir las ecuaciones linealizadas y resolver el sistema mediante una formulación explícita derivada de las expresiones anteriores.

La función `trilaterate_3D` implementa la resolución directa del sistema para tres anclas, utilizando la ancla 1 como referencia; como se puede ver en el listado 2 de la sección de Anexos. El algoritmo obtiene las constantes intermedias A, B, C, D, E, F, G, H correspondientes a las dos ecuaciones independientes, y a partir de ellas se construye un sistema auxiliar para despejar el eje z :

$$\begin{cases} Ax + By + Cz = D \\ Ex + Fy + Gz = H \end{cases}$$

De este sistema se derivan las relaciones:

$$z = \frac{-BG + CF}{AF - EB}x + \frac{-BH + DF}{AF - EB}, \quad y = \frac{AG - EC}{AF - EB}x + \frac{AH - ED}{AF - EB}$$

Sustituyendo en la ecuación del primer sensor se obtiene una ecuación cuadrática en z , evaluando el discriminante $\Delta = b^2 - 4ac$:

- Si $\Delta > 0$, el sistema devuelve la primera raíz real para z .
- Si $\Delta < 0$, se asigna $z = 0$, evitando la propagación de errores al plano horizontal.

Extensión a multilateración y resolución mediante ecuaciones normales

Si bien el modelo de trilateración permitió determinar la posición en dos dimensiones utilizando tres sensores, su desempeño dependía fuertemente de la precisión de las mediciones individuales. Para mitigar el impacto del ruido y mejorar la estabilidad del posicionamiento, se implementó una extensión del modelo basada en multilateración, que permite aprovechar simultáneamente más de tres sensores (hasta seis en las pruebas realizadas).

En este caso, el sistema se expresa de forma general como:

$$\begin{bmatrix} A_{1x} & A_{1y} & A_{1z} \\ A_{2x} & A_{2y} & A_{2z} \\ \vdots & \vdots & \vdots \\ A_{n-1x} & A_{n-1y} & A_{n-1z} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} A_{1d} \\ A_{2d} \\ \vdots \\ A_{n-1d} \end{bmatrix}$$

que puede escribirse compactamente como:

$$A\mathbf{x} = \mathbf{b}$$

Cuando el número de sensores n es mayor a tres, el sistema se vuelve sobredeterminado —es decir, no existe un único vector $\mathbf{x}$ que satisfaga exactamente todas las ecuaciones simultáneamente debido a errores de medición y ruido. En este escenario, la estrategia consiste en encontrar la mejor aproximación en el sentido de los mínimos cuadrados, lo cual se consigue resolviendo la ecuación normal asociada:

$$A^T A\mathbf{x} = A^T \mathbf{b}$$

Derivación de la ecuación normal

Sea el sistema $A\mathbf{x} = \mathbf{b}$ con $A \in \mathbb{R}^{m \times n}$ y $m > n$. Dado que el sistema es inconsistente, $\mathbf{b}$ no pertenece al espacio columna de A ($\mathbf{b} \notin \text{col}(A)$). Sin embargo, existe un vector $\hat{\mathbf{x}}$ tal que $A\hat{\mathbf{x}}$ representa la proyección ortogonal de $\mathbf{b}$ sobre $\text{col}(A)$, minimizando el error cuadrático:

$$A\hat{\mathbf{x}} = \text{proj}_{\text{col}(A)}(\mathbf{b})$$

$$\mathbf{b} - A\hat{\mathbf{x}} = \mathbf{b} - \text{proj}_{\text{col}(A)}(\mathbf{b})$$

Multiplicando ambos lados por A^T se obtiene:

$$A^T(\mathbf{b} - A\hat{\mathbf{x}}) = 0$$

$$A^T A\hat{\mathbf{x}} = A^T \mathbf{b}$$

Esta formulación corresponde al sistema de ecuaciones normales, cuya solución $\hat{\mathbf{x}}$ representa el punto que minimiza la suma de los errores al cuadrado entre las distancias medidas y las distancias modeladas. Geométricamente,

esto equivale a proyectar el vector $\mathbf{b}$ sobre el subespacio generado por las columnas de A , como se muestra en la Figura 1.

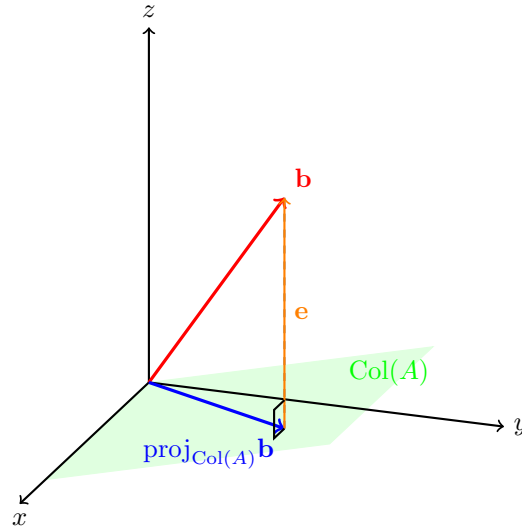


Figura 1: Proyección del vector $\mathbf{b}$ sobre el espacio columna de A , base conceptual de las ecuaciones normales.

Implementación de multilateración en Swift

La multilateración se implementó mediante la función `multilateration()` y se puede observar en el listado 3, que construye las matrices del sistema y aplica las ecuaciones normales para resolver el vector de posición en tiempo real. El proceso se ejecuta en tres etapas:

1. **Construcción de matrices:** La función `multInitializeMatrixVector()` genera la matriz A y el vector $\mathbf{b}$ en base a las posiciones y distancias de cada ancla, tomando una de ellas como referencia o *pivot*.
2. **Cálculo de las ecuaciones normales:** Se obtiene el producto $A^T A$ y $A^T \mathbf{b}$ mediante multiplicación matricial, creando un sistema cuadrado de dimensión 3×3 .
3. **Resolución del sistema:** El sistema resultante se resuelve mediante el método de Gauss-Jordan, devolviendo el vector de posición (x, y) .

Esta formulación basada en ecuaciones normales proporciona una estimación más robusta de la posición, al reducir la sensibilidad del sistema frente a errores individuales en las mediciones de distancia. En las pruebas experimentales, la multilateración permitió estabilizar significativamente el cálculo de posición respecto al modelo de trilateración pura, generando trayectorias más suaves y coherentes con el movimiento real del usuario.

Es importante destacar que la formulación de multilateración mediante el sistema $A\mathbf{x} = \mathbf{b}$ asume, en teoría, que la matriz A es linealmente independiente (LI), lo cual garantiza una solución única y exacta para la posición del usuario.

Sin embargo, durante las pruebas en el entorno universitario se observó que la disposición de los sensores generaba situaciones en las que algunos beacons estaban prácticamente alineados unos detrás de otros. En estos casos, la matriz A se volvía linealmente dependiente (LD), impidiendo la obtención de una solución directa mediante el solver exacto de Gauss-Jordan.

Para resolver esta limitación se implementó un *least squares solver* basado en las ecuaciones normales, es decir, se resolvió $A^T A \mathbf{x} = A^T \mathbf{b}$. Esta aproximación no requiere que A sea LI y permite calcular la posición que minimiza la suma de errores cuadrados respecto a todas las mediciones disponibles.

Esta estrategia demostró ser altamente conveniente por dos razones: primero, el solver utilizado (implementado con funciones de LAPACK integradas en Swift) está optimizado para cálculos matriciales y resultó significativamente más rápido que una implementación propia desde cero; segundo, permitió aprovechar simultáneamente más de tres sensores, mejorando la robustez del posicionamiento y la suavidad de las trayectorias, incluso en situaciones donde la matriz A era casi singular.

En resumen, aunque la linealidad de A es un requisito teórico ideal, la aplicación práctica del solver de mínimos cuadrados permitió superar las limitaciones impuestas por la geometría de los sensores en el entorno real, garantizando resultados precisos y un rendimiento adecuado para la experiencia del usuario.

9.1.3. Fase 3: Validación funcional

Validación experimental y observaciones

Una vez implementado el algoritmo de trilateración en 2D, se realizaron pruebas controladas con tres sensores Estimote UWB para evaluar su funcionamiento. La primera prueba se llevó a cabo en un salón del campus universitario, con el objetivo de obtener una visualización inicial del posicionamiento calculado. En esta etapa se generó una animación (GIF) que mostraba en tiempo real la estimación de la ubicación del usuario.

En dichas pruebas preliminares se observó que las posiciones estimadas presentaban una variabilidad considerable, aun cuando el dispositivo se mantenía estático. Esta fluctuación —visible como movimientos erráticos del punto de localización en la animación— evidenció la presencia de ruido significativo en las mediciones de distancia obtenidas de los sensores.

Posteriormente, se realizó una segunda fase de pruebas en un entorno controlado, específicamente en un cuarto rectangular donde los sensores se dispusieron a una misma altura y en posiciones fijas previamente medidas. En este escenario se registraron las coordenadas calculadas por la trilateración a lo largo del tiempo, mientras el usuario se desplazaba en diagonal entre el origen y el primer cuadrante del plano de referencia.

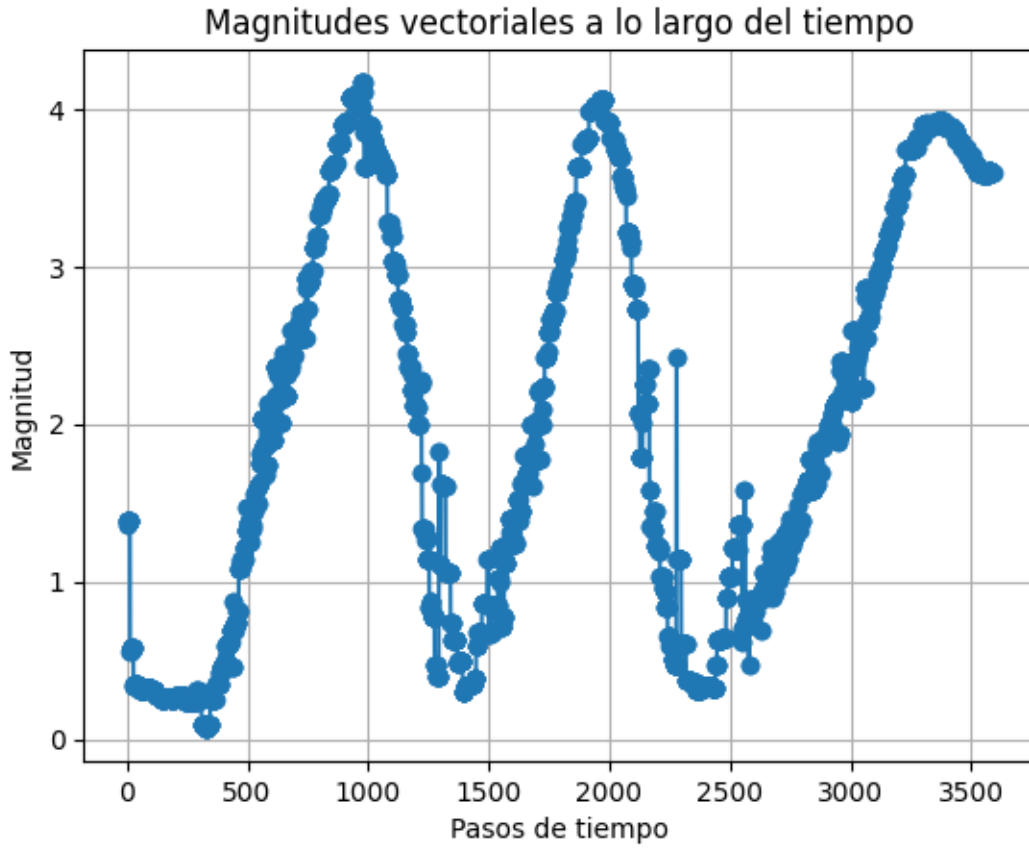


Figura 2: Gráfica de la magnitud del vector de posición a lo largo del tiempo, mostrando el jitter observado durante la trilateración 2D.

Para analizar cuantitativamente las variaciones de posición, se calculó la magnitud del vector de posición $\sqrt{x^2 + y^2}$ y se graficó su evolución temporal. Dado que el movimiento real seguía una trayectoria diagonal y uniforme, se esperaba que la magnitud de la posición oscilara entre 0 y un valor máximo positivo, describiendo un patrón aproximadamente sinusoidal a lo largo del tiempo. Sin embargo, la gráfica resultante (Figura 2) mostró oscilaciones abruptas e irregulares superpuestas a esta tendencia esperada, confirmando la presencia de jitter y ruido de alta frecuencia en las estimaciones de posición.

Estos resultados demostraron que, aunque la trilateración permitía calcular posiciones válidas en el plano x,y, la señal obtenida era altamente inestable y no adecuada para su uso directo en una aplicación de realidad aumentada sin aplicar técnicas adicionales de filtrado o suavizado.

9.2. Objetivo 2: Integrar el SDK nativo de Estimote con Unity mediante un plugin de comunicación en tiempo real

Este objetivo tuvo como finalidad establecer la comunicación entre los sensores UWB de Estimote y el entorno Unity, mediante el desarrollo de un módulo nativo en Swift y su integración con la aplicación de realidad aumentada. A través de este proceso se buscó habilitar la adquisición de datos en tiempo real desde el hardware, garantizando su transferencia fluida hacia el motor de visualización y procesamiento.

Fase 1: Conexión con sensores UWB

Durante esta etapa se realizaron múltiples pruebas con los diferentes SDKs oficiales de Estimote. En un principio se experimentó con los SDKs *Indoor Location* y *Proximity*, los cuales en teoría ofrecían funcionalidades equivalentes a un sistema GPS para interiores. Sin embargo, en la práctica ambos presentaron limitaciones técnicas significativas: requerían una configuración extensa, la conexión a los sensores resultaba altamente inestable y el control disponible sobre los dispositivos era insuficiente para los requerimientos de la aplicación.

El *Proximity SDK* resultaba especialmente prometedor, dado que ofrecía la detección automática de balizas y la generación de eventos de proximidad, pero su comportamiento inconsistente y la limitada documentación disponible lo hicieron inviable para un entorno de desarrollo robusto. Estas pruebas iniciales evidenciaron la necesidad de un enfoque más directo y flexible.

Posteriormente se optó por emplear otro SDK de Estimote que permitía una interacción de menor nivel con los sensores. Este proporcionaba acceso directo a las mediciones de distancia tan pronto como el sensor era descubierto, permitiendo la obtención continua de actualizaciones en tiempo real. A pesar de la mayor libertad que ofrecía, este SDK también presentaba limitaciones importantes: no permitía seleccionar manualmente a qué sensores conectarse, y una vez que un sensor era descubierto, si no se establecía conexión inmediatamente, ya no era posible hacerlo hasta que el sensor se perdiera completamente del rango de detección. Asimismo, aunque el SDK incluía funciones para listar los sensores cercanos, muchas de ellas no funcionaban conforme a la documentación oficial.

Una funcionalidad particularmente interesante de este SDK era la posibilidad de obtener, además de la distancia, el ángulo del sensor respecto al dispositivo móvil. Con estos dos valores —distancia y ángulo— sería posible determinar la posición del dispositivo con un único sensor, eliminando la necesidad de trilateración. Sin embargo, esta capacidad solo estaba disponible en un conjunto reducido de dispositivos con múltiples antenas UWB. De acuerdo con la información obtenida en foros de la comunidad de Estimote (ante la escasa documentación oficial), esta función era compatible únicamente con modelos de iPhone 11, 12 y 13. Durante las pruebas se confirmó este comportamiento: un iPhone 12 Mini logró reportar el ángulo correctamente, mientras que modelos más recientes, como el iPhone 14 y 16, no proporcionaban dicha información.

Este hallazgo resultó determinante en el diseño final del sistema. Dado que la obtención del ángulo no era una funcionalidad universal, se descartó el enfoque de posicionamiento basado en un solo sensor y se optó por la trilateración. Esta decisión, aunque más robusta, implicó la necesidad de utilizar al menos tres sensores UWB de manera simultánea, triplicando la cantidad de hardware inicialmente considerada por los antecesores del proyecto.

Discusión sobre el enfoque previo del proyecto

Antes de adoptar el nuevo enfoque, se realizó una prueba de concepto basada en la lógica planteada por el equipo previo del megaproyecto. Esta consistía en colocar un único sensor UWB en puntos clave del recorrido —por ejemplo, cruces o intersecciones— con el propósito de guiar al usuario mediante la brújula del dispositivo. La idea era que, al conectarse al sensor más cercano, el sistema pudiera deducir el punto actual del recorrido y orientar al usuario hacia la siguiente dirección.



Figura 3: Prueba de concepto con enfoque inicial basada en un solo sensor UWB.

Aunque funcional en escenarios simples, este enfoque presentó múltiples limitaciones para su uso en producción. La conexión con los sensores resultó poco confiable: incluso situando el dispositivo directamente frente al sensor, la conexión podía tardar varios segundos o, en algunos casos, no establecerse en absoluto. A mayores distancias, el problema se agravaba. Además, la escalabilidad del sistema era prácticamente nula, ya que cada nuevo *landmark* agregado requería instalar físicamente un sensor adicional y modificar la capa de software correspondiente. Esto hacía el mantenimiento del sistema costoso y poco sostenible.

Por estas razones, se decidió descartar este enfoque y migrar hacia un sistema basado en trilateración, utilizando tres o más sensores conectados simultáneamente para estimar la posición. Para mitigar los problemas de conexión, se implementó la posibilidad de mantener hasta seis conexiones activas en paralelo, proporcionando redundancia y mayor estabilidad en la adquisición de datos.

Fase 2: Desarrollo del plugin nativo para Unity

Con la comunicación nativa ya establecida, el siguiente paso consistió en desarrollar un plugin que actuara como puente entre Swift (iOS) y Unity. La elección de Unity se basó en su capacidad de ofrecer un entorno multiplataforma, permitiendo construir una sola interfaz visual y mantener únicamente la capa de conexión nativa diferenciada para iOS y Android.

Técnicamente, la integración se realizó exponiendo funciones Swift mediante el sistema de interoperabilidad de Objective-C con Swift, asegurando que fueran públicas para que Unity pudiera reconocerlas en tiempo de ejecución.

Este mecanismo permitió que la capa lógica escrita en Swift permaneciera aislada, mientras que Unity se encargaba de visualizar los datos procesados en tiempo real.

Fase 3: Validación del flujo de comunicación

La integración final se validó mediante una escena de prueba en Unity en la que un objeto tridimensional se desplazaba de acuerdo con las coordenadas calculadas por el módulo nativo. En esta arquitectura, Unity ejecuta llamadas periódicas a las funciones expuestas del plugin para obtener los valores más recientes de posición. Dado que la comunicación es unidireccional —de Unity hacia Swift—, no se implementaron mecanismos de notificación inversa. En su lugar, Unity consulta el estado del sistema en cada actualización de cuadro (*frame*), lo cual es suficiente dado el carácter continuo de la actualización de datos.

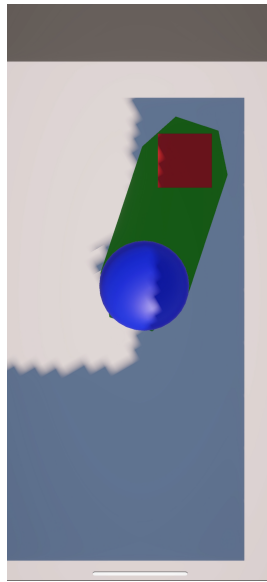


Figura 4: Prueba de concepto mediante escena de Unity, en azul el objeto tridimensional representando el dispositivo, en verde la ruta que debe seguir el objeto, en rojo la posición de destino.

Este diseño logró un desempeño fluido e indistinguible de una aplicación nativa, manteniendo frecuencias de actualización sin interrupciones visibles. La comunicación entre el hardware, el módulo nativo y Unity demostró ser estable y de baja latencia, cumpliendo con los requerimientos para la visualización de datos en tiempo real dentro del entorno de realidad aumentada.

9.3. Objetivo 3: Aplicación del filtro de Kalman para reducir el jitter y fusionar datos sensoriales

La trilateración implementada previamente permitió obtener coordenadas de posición bidimensional del usuario, pero las mediciones resultaron demasiado inestables para su uso en una aplicación de producción. Las posiciones fluctuaban constantemente, incluso cuando el dispositivo permanecía estático, lo cual se debía al ruido intrínseco de los sensores UWB. Para mitigar este problema, se propuso combinar la información de la trilateración con los datos inerciales del acelerómetro del dispositivo móvil, aplicando un filtro de Kalman como técnica de fusión y suavizado de señales.

Fase 1: Recolección de datos crudos

La lógica detrás de esta etapa era sencilla: si el acelerómetro indicaba que el usuario no se estaba moviendo, pero las coordenadas de la trilateración mostraban variaciones abruptas, entonces estas oscilaciones debían atribuirse al ruido de medición. Bajo este principio, se decidió capturar datos simultáneamente de ambos sistemas: las posiciones obtenidas mediante trilateración y las aceleraciones registradas por el acelerómetro.

Inicialmente se evaluó la posibilidad de calcular la posición únicamente a partir de los datos del acelerómetro, integrando dos veces la aceleración en el tiempo. Sin embargo, este enfoque se descartó rápidamente, ya que los errores de integración se acumulaban exponencialmente, provocando un crecimiento descontrolado en la magnitud del vector de velocidad y posición. En pruebas realizadas desplazándose en diagonal entre el origen y un punto máximo en el primer cuadrante, se observó que la magnitud del vector velocidad aumentaba de forma irreal, en lugar de mantenerse en valores compatibles con la velocidad promedio de una persona caminando (Figura 5).

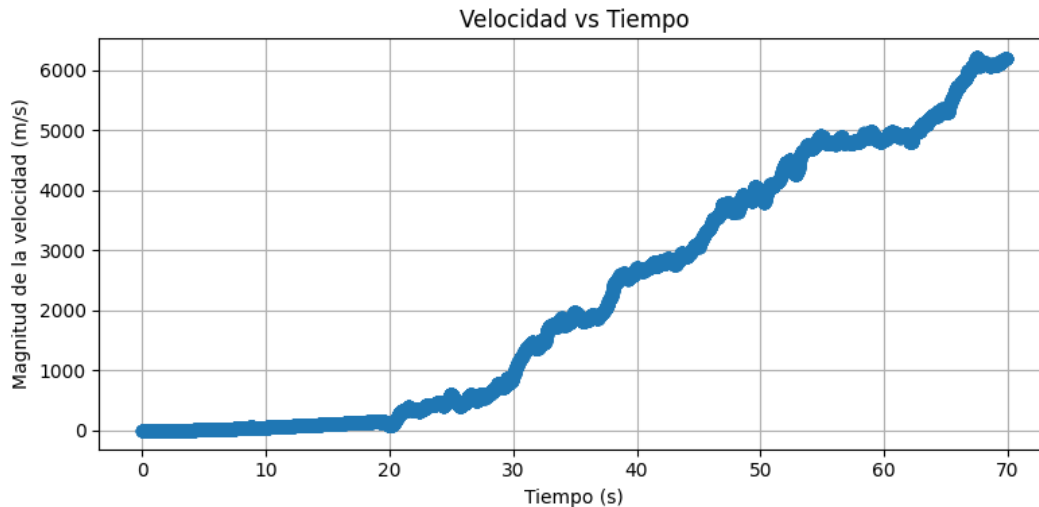


Figura 5: Crecimiento de la velocidad al integrar los datos del acelerómetro para estimar la posición.

Ante estos resultados, se optó por fusionar ambos conjuntos de datos mediante un filtro de Kalman. Este método no solo permite combinar información de distintas fuentes bajo un modelo dinámico de movimiento, sino que además suaviza el ruido de las mediciones. En la práctica, el filtro aprende progresivamente cómo se comporta el sistema, de modo que sus predicciones se vuelven más confiables que las propias mediciones crudas, las cuales solo se utilizan para realizar correcciones incrementales.

Para garantizar la reproducibilidad y evitar depender de pruebas físicas en cada iteración, se decidió capturar los datos crudos en un entorno controlado y analizarlos de forma simulada. Los datos fueron recolectados en el mismo cuarto rectangular utilizado para las pruebas de trilateración, desplazándose nuevamente en diagonal entre el origen y el primer cuadrante. Se almacenaron tanto las posiciones estimadas como las lecturas del acelerómetro en un archivo de texto con el siguiente formato:

```
Time: 1750964200.546540, Accel: x:-0.002765, y:-0.004459, z:-0.000987
Timestamp: 1750964200.603205, Position: Coordinate(x: 1.0589412, y: -0.8869535, z: 0.0)
```

Dado que el archivo también contenía otros registros de la aplicación, fue necesario desarrollar un script de limpieza y parseo en Python, encargado de ordenar los datos cronológicamente y normalizar los formatos de tiempo y magnitud antes de alimentar la simulación del filtro de Kalman.

Fase 2: Implementación y ajuste del filtro de Kalman

La simulación del filtro de Kalman se desarrolló inicialmente en Python, lo que permitió experimentar rápidamente con los hiperparámetros y evaluar los resultados visualmente. Antes de aplicar el filtro, se calculó la varianza de los datos crudos con el fin de estimar los valores iniciales de ruido de medición (R) y ruido del proceso (Q), fundamentales en el ajuste del filtro.

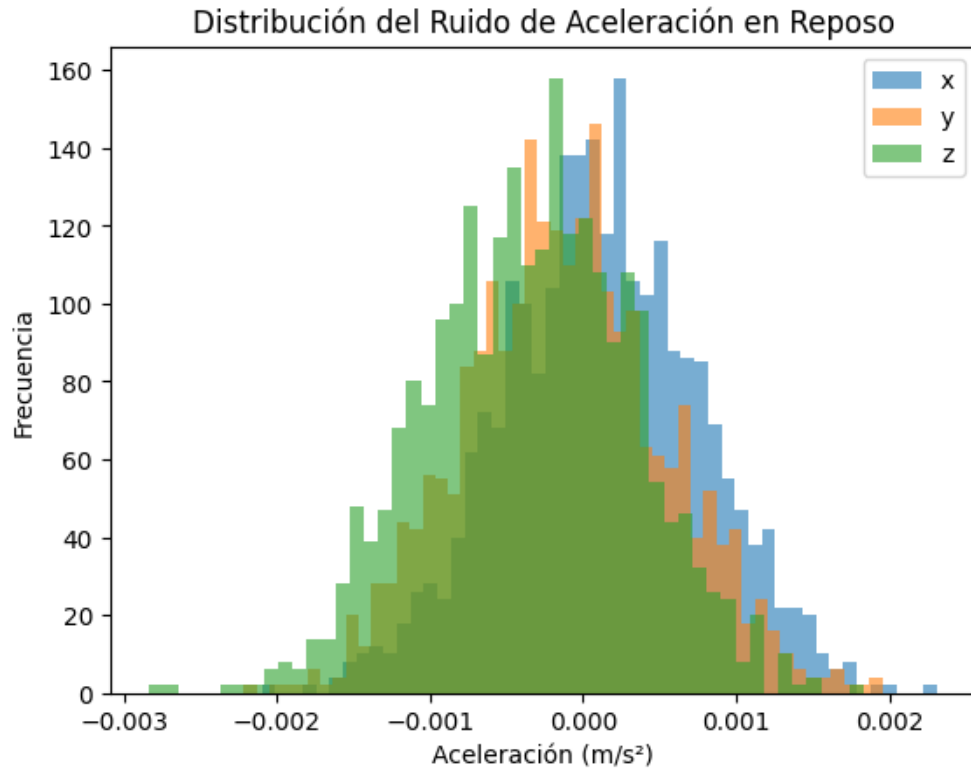


Figura 6: Distribución del acelerómetro en reposo registrada mediante *CoreMotion*. Los tres ejes presentan una varianza mínima, consistente con un estado de inmovilidad del dispositivo.

La varianza de las lecturas del acelerómetro en reposo fue extremadamente baja:

$$\text{Var}_{acc} = [4,16 \times 10^{-7}, 4,33 \times 10^{-7}, 4,52 \times 10^{-7}]$$

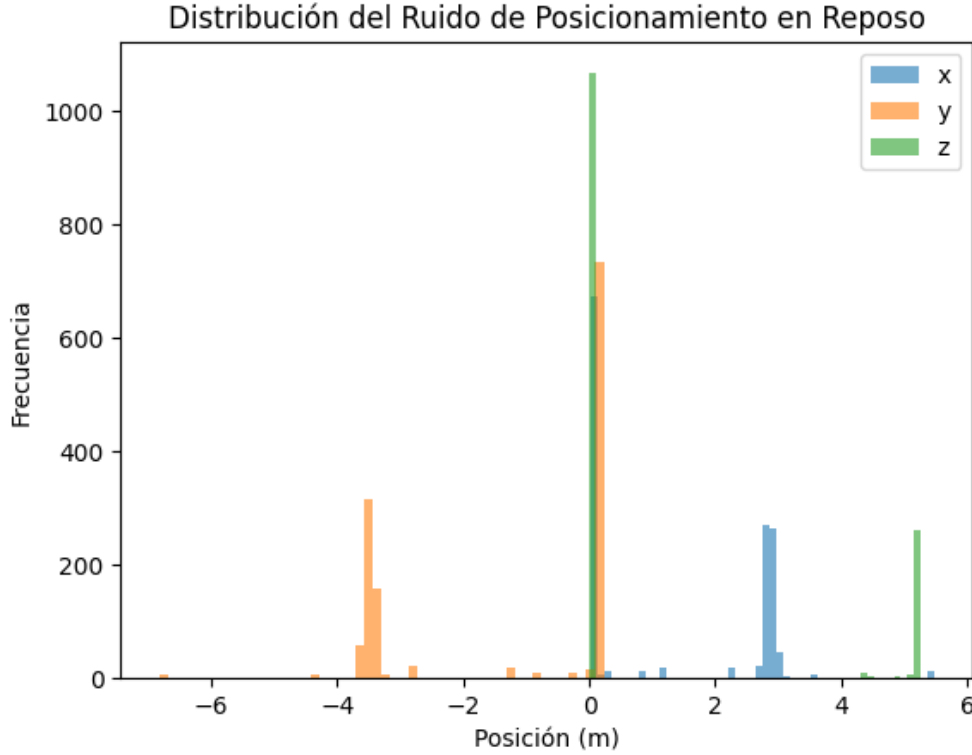


Figura 7: Distribución de la posición estimada mediante trilateración en reposo. Se observan los tres ejes superpuestos.

Mientras que la varianza de los datos de posicionamiento obtenidos por trilateración resultó significativamente mayor:

$$\text{Var}_{pos} = [2,02, 3,23, 4,40]$$

El eje z presentó una varianza más elevada debido a la naturaleza arbitraria de su cálculo (recordando que no siempre se encontraba una solución real al sistema, y en caso contrario se asignaba $z = 0$). Esta diferencia era esperada, ya que el sistema no buscaba exactitud en z , sino estabilidad en el plano x, y .

Aunque los valores anteriores ofrecían una referencia inicial, se observó que las condiciones estáticas del acelerómetro subestimaban la varianza real durante el movimiento. Por ello, los hiperparámetros del filtro se determinaron finalmente de forma empírica, ajustándolos iterativamente hasta lograr un equilibrio entre respuesta rápida y suavizado efectivo.

El resultado se validó mediante la comparación visual entre la magnitud del vector de posición filtrado y no filtrado a lo largo del tiempo.

Para ilustrar de manera más intuitiva el efecto del filtro de Kalman sobre el movimiento, se generó una trayectoria diagonal del usuario dentro del entorno de prueba. La Figura 8 muestra en el mismo plano la posición estimada con y sin filtrado, permitiendo observar claramente cómo el filtro suaviza los desplazamientos y reduce los saltos erráticos presentes en los datos crudos.

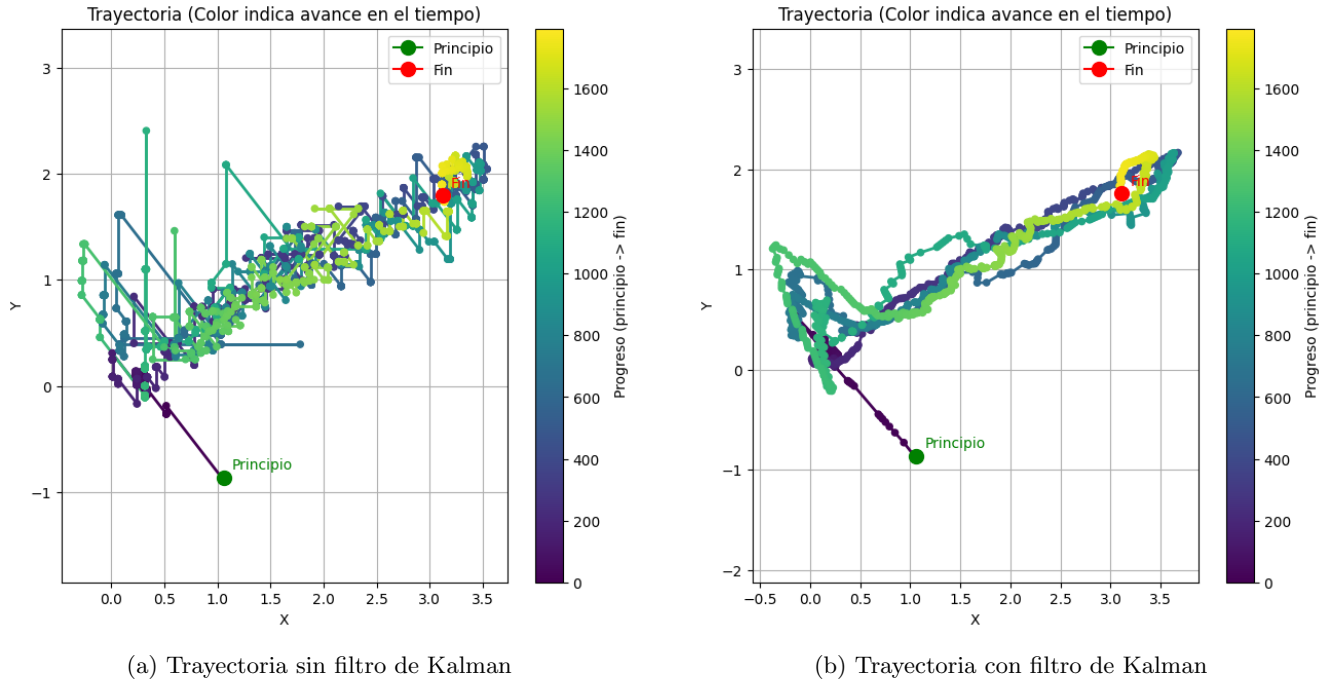


Figura 8: Comparación de la trayectoria diagonal del usuario en movimiento. La trayectoria filtrada muestra reducción de jitter y mayor suavidad respecto a la señal sin filtrar.

Mientras la señal original mostraba un patrón ruidoso con picos abruptos (Figura 2), la señal filtrada exhibía una forma sinusoidal limpia y continua (Figura 9), reflejando el movimiento diagonal esperado.

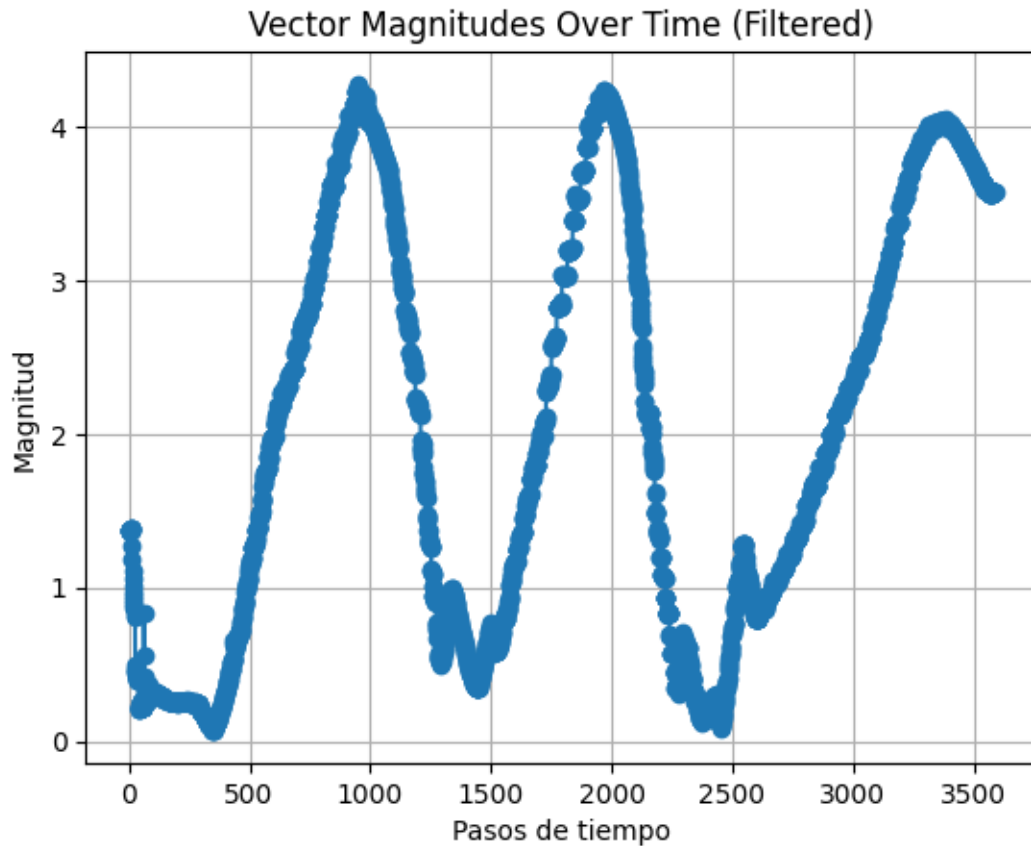


Figura 9: Magnitudes de posición filtradas con Kalman.

Fase 3: Migración a Swift y refactorización arquitectónica

Una vez calibrado el filtro en Python, se procedió a su implementación en Swift dentro del plugin nativo. En este punto fue necesario realizar una refactorización completa de la arquitectura del sistema, que hasta ese momento había sido utilizada únicamente como entorno experimental.

El nuevo diseño adoptó el patrón de delegación (*delegate pattern*) para desacoplar los módulos principales del sistema: gestión de sensores UWB, adquisición de datos inerciales y procesamiento de posición. Esta modularización mejoró la legibilidad del código y facilitó la extensión futura del sistema a otras plataformas.

La arquitectura final se estructuró en torno a tres componentes principales:

- **UWBManager**: encargado de gestionar la conexión y actualización de distancias provenientes de los sensores UWB.
- **AccelerometerManager**: responsable de recolectar y transmitir las lecturas del acelerómetro usando *CoreMotion*.
- **PositionCoordinator**: módulo central que fusiona las fuentes de datos aplicando la trilateración y el filtro de Kalman, devolviendo las coordenadas finales del usuario.

Estos módulos operan en un hilo secundario dedicado, con el fin de no bloquear el *main thread* responsable de la interfaz gráfica. La decisión de mantener todo el procesamiento de localización en un solo hilo secundario obedeció al deseo de evitar sobrecargas por sincronización o calendarización, dado que el rendimiento observado era suficiente para un procesamiento en tiempo real.

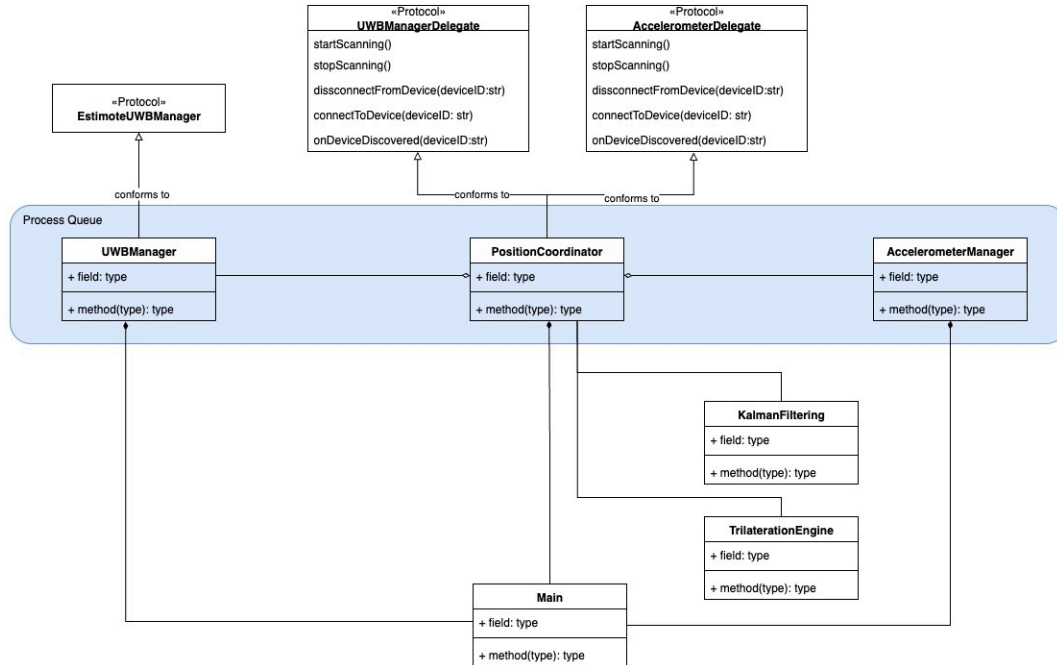


Figura 10: Arquitectura final del sistema tras la integración del filtro de Kalman.

9.4. Objetivo 4: Documentación del desarrollo e integración del plugin

Fase 1: Estructuración del código y módulos

Durante esta fase se organizó el código fuente del plugin nativo en **Swift** siguiendo una arquitectura modular y de única responsabilidad. Cada componente fue diseñado para cumplir una función específica dentro del flujo general del sistema: **UWBManager** gestiona las conexiones y lecturas de los sensores UWB, **AccelerometerManager** se encarga de capturar y normalizar los datos inerciales del dispositivo, y **PositionCoordinator** actúa como núcleo del cálculo de posición, combinando la información proveniente de los módulos anteriores y enviándola a Unity.

Esta separación de responsabilidades permitió mantener un código limpio, fácil de depurar y con bajo acoplamiento entre módulos, favoreciendo la posibilidad de extender o sustituir componentes sin afectar al resto del sistema. Asimismo, se establecieron convenciones de nomenclatura consistentes y se documentó internamente cada clase, propiedad y método con descripciones claras de su funcionalidad.

Fase 2: Elaboración de documentación técnica

Toda la documentación del plugin se generó en el formato nativo de Apple, **.docc** (*Documentation Catalogs*), el cual permite integrar la documentación directamente en Xcode y consultarla mediante la herramienta de documentación oficial de la plataforma. Este formato ofrece ventajas como navegación estructurada por símbolos, vínculos automáticos entre tipos y métodos, y compatibilidad con el sistema de generación de documentación de Swift.

Cada función, método y clase del plugin incluye una descripción formal, la explicación de sus parámetros y valores de retorno, así como notas de uso y ejemplos breves. Esto facilita a futuros desarrolladores comprender el flujo del sistema sin necesidad de inspeccionar el código fuente en detalle.

Además de la documentación integrada, se elaboró un archivo **README.md** en el repositorio principal. Este documento describe el proceso de compilación del plugin, su integración con Unity, y proporciona un ejemplo funcional que muestra cómo invocar los métodos nativos desde **C#**. Se incluyen también los requisitos de entorno y dependencias necesarias para su ejecución, lo que convierte al repositorio en un punto de partida completo para desarrolladores que deseen extender o mantener el sistema.

Fase 3: Publicación y entrega

Con el fin de asegurar la continuidad del proyecto, toda la documentación —incluyendo el catálogo **.docc**, el archivo **README.md**, y los diagramas de arquitectura— fue almacenada dentro del mismo repositorio Git que contiene el código fuente. Esto garantiza que la documentación evolucione junto al software, evitando la pérdida de información técnica en futuras iteraciones.

Asimismo, todas las descripciones, comentarios y mensajes de commit en el repositorio fueron redactados en inglés. Esta decisión busca mantener la coherencia con las convenciones internacionales de desarrollo y garantizar la accesibilidad del proyecto a colaboradores externos. Desde una perspectiva de ingeniería de software, el uso del inglés en documentación técnica y control de versiones constituye una buena práctica, ya que promueve la estandarización y facilita la comprensión por parte de equipos distribuidos.

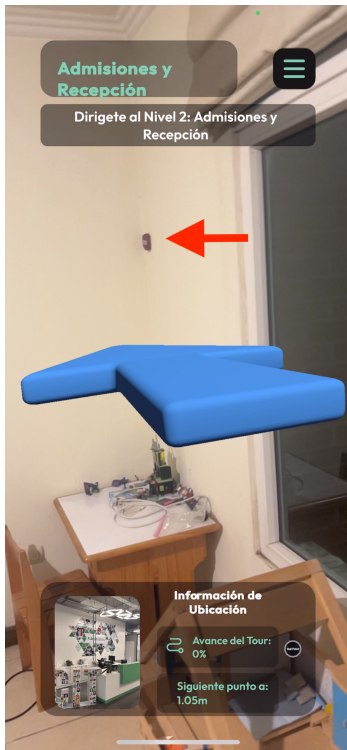
Finalmente, la documentación técnica y el código serán entregados como parte integral del megaproyecto de realidad aumentada, asegurando su preservación y disponibilidad para futuras generaciones de estudiantes y desarrolladores. Con ello, se consolida una base estructurada y documentada que permitirá continuar la evolución del sistema con un menor costo de aprendizaje y mayor consistencia entre versiones.

9.5. Objetivo 5: Evaluar cualitativamente la precisión y suavidad del sistema de localización

Fase 1: Pruebas de campo

Las primeras pruebas de campo tuvieron como propósito observar el comportamiento del sistema de localización tanto en reposo como en movimiento, verificando la coherencia visual entre la posición estimada del usuario y la dirección mostrada en la aplicación de realidad aumentada. Inicialmente se efectuó una prueba de concepto en un entorno controlado —una habitación— en la que se dispusieron cuatro sensores UWB (beacons) de Estimote colocados en disposición rectangular. En este entorno se probó la integración completa entre el sistema de multilateración, el filtro de Kalman y la aplicación desarrollada en Unity.

Durante las pruebas se observó el mapa desde vista superior para visualizar el posicionamiento del jugador en el entorno virtual y posteriormente se evaluó la flecha de realidad aumentada que indica la dirección del objetivo. En todos los casos, la flecha apuntó de forma consistente hacia el objetivo y los movimientos fueron suaves y estables, confirmando la correcta fusión entre el algoritmo de multilateración y el filtro de Kalman. No se detectaron comportamientos erráticos ni fluctuaciones visuales significativas.



(a) Vista de la flecha apuntando al objetivo correctamente (señalado en rojo).



(b) Vista de la flecha apuntando al objetivo correctamente (señalado en rojo).

Figura 11: Pruebas domésticas con el sistema completamente integrado. Las trayectorias muestran un movimiento continuo y estable, confirmando la correcta sincronización entre el algoritmo de trilateración, el filtro de Kalman y la aplicación de realidad aumentada.

A partir de estos resultados iniciales, se trasladaron las pruebas al entorno real de la Universidad del Valle de Guatemala, específicamente al sexto nivel del edificio. En esta etapa se utilizaron cinco sensores, distribuidos según

las coordenadas obtenidas experimentalmente:

```
{  
  "44d22f737b259dd4e31a23b324ef941f": { "x": 12.95, "y": 4.49, "z": 2.5 },  
  "550aed08145ffba276256b188a968124": { "x": 16.99, "y": -2.89, "z": 2.5 },  
  "420139767c96fee20dec018c2ec7a33a": { "x": 19.26, "y": 4.99, "z": 2.5 },  
  "dc89a1ea448ecf9b5c8f9f66af1ecc06": { "x": 13.03, "y": 12.2, "z": 2.5 },  
  "fe08fa755807f79abf1a233724ed182e": { "x": 14.58, "y": 22.94, "z": 2.5 }  
}
```

Los sensores se encontraban instalados a una altura de aproximadamente 2.5 metros, separados entre 10 y 15 metros. Durante esta etapa se observaron múltiples dificultades en la conectividad con los sensores, ya que el dispositivo móvil raramente lograba mantener conexión con más de tres sensores simultáneamente. Además, los tiempos de actualización eran inconsistentes, lo que generaba saltos abruptos en la posición estimada.

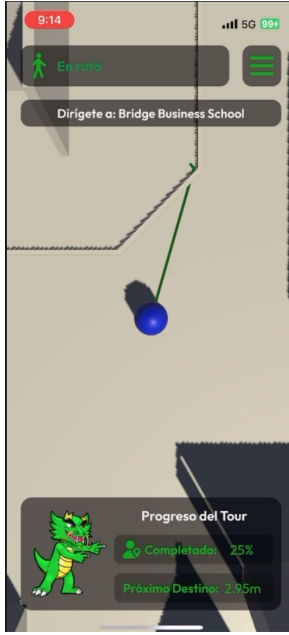
Estas limitaciones llevaron a plantear tres hipótesis principales: (1) interferencia electromagnética causada por la gran cantidad de señales BLE, Wi-Fi y dispositivos electrónicos en el campus; (2) desgaste en las baterías de los sensores, adquiridos un año antes; y (3) exceso de separación y altura de instalación, lo que reducía la calidad de la señal recibida por el dispositivo móvil.

Fase 2: Recolección y análisis de datos

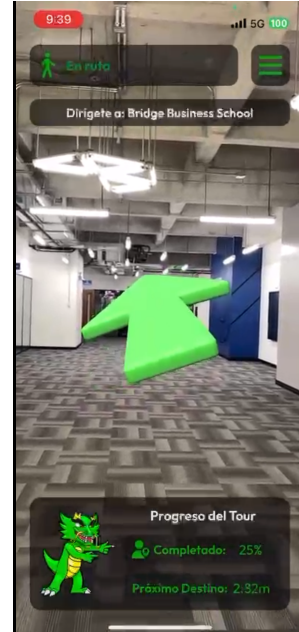
Con el fin de analizar la estabilidad visual y la suavidad de movimiento, se registraron observaciones cualitativas del sistema en funcionamiento. En las pruebas realizadas en el entorno doméstico, la estabilidad visual fue óptima, por lo que no se requirieron modificaciones adicionales ni al filtro de Kalman ni al algoritmo de multilateración.

En contraste, las pruebas en el entorno universitario presentaron un movimiento inconsistente. En varios casos el marcador virtual permanecía estable durante algunos segundos, pero luego sufría desplazamientos erráticos o retrasos notables en la actualización de posición. Este comportamiento se atribuyó a la falta de sincronización en las actualizaciones de distancia, ya que en múltiples ocasiones únicamente se recibían datos de uno o dos sensores, impidiendo una trilateración precisa.

Los datos de posicionamiento obtenidos en estas pruebas se emplearán para generar gráficas comparativas que reflejen el jitter y la dispersión espacial del sistema.



(a) Vista superior del mapa con la trayectoria del usuario en el entorno universitario.



(b) Vista en primera persona desde la aplicación de realidad aumentada, mostrando la flecha direccional hacia el objetivo.

Figura 12: Pruebas en el entorno universitario mostrando la trayectoria del usuario (izquierda) y la visualización en realidad aumentada (derecha). Ambas perspectivas confirman la coherencia entre la posición estimada y la dirección mostrada al usuario.

Asimismo, se incluirán capturas de pantalla representativas del comportamiento visual del sistema durante las pruebas, con el fin de complementar el análisis cualitativo.

Fase 3: Ajustes iterativos

Como parte del proceso de mejora, se procedió a reemplazar las baterías de los sensores, reducir la distancia entre ellos y disminuir la altura de instalación a aproximadamente 1.65 metros. Estas modificaciones mejoraron parcialmente la estabilidad de la conexión, lo que permitió continuar con nuevas pruebas.

Adicionalmente, se identificó una limitación técnica importante en el *Software Development Kit* (SDK) de Estimote, utilizado para gestionar las conexiones con los sensores desde la aplicación desarrollada en Swift e integrada posteriormente con Unity. Inicialmente, el sistema mantenía un límite de seis conexiones simultáneas, suficiente para realizar trilateración con redundancia; sin embargo, esto impedía conectar sensores adyacentes y obstaculizaba la continuidad del recorrido.

Para mitigar esta restricción, el límite de conexiones se aumentó a 100, lo cual generó un nuevo error interno de calendarización de hilos (*EXC_BAD_ACCESS*). Dicho error provenía directamente del ensamblado del SDK propietario, impidiendo su depuración o modificación. Aunque se intentó realizar un *rollback* a versiones anteriores del plugin, e incluso se probó el SDK de forma aislada, el error persistía al conectar más de seis sensores.

Es importante destacar que este fallo no se encontraba en la implementación del algoritmo de trilateración ni en el filtro de Kalman, sino en la capa de comunicación proporcionada por el SDK.

El SDK de Estimote, al ser software propietario distribuido como binario compilado para Swift, no expone su

código fuente ni símbolos de depuración. Esto impide acceder a los procesos internos de manejo de hilos, colas de actualización o temporizadores, limitando la posibilidad de identificar el origen exacto del error o aplicar correcciones a nivel de código. Incluso la aplicación oficial de Estimote presentaba fallas similares al intentar gestionar múltiples conexiones simultáneas, lo cual refuerza la hipótesis de que se trata de una deficiencia estructural del SDK.

Finalmente, el sistema logró mantener un comportamiento estable en entornos controlados, confirmando la efectividad del modelo de filtrado y posicionamiento. No obstante, las limitaciones técnicas del SDK imposibilitaron una evaluación más amplia en entornos reales.

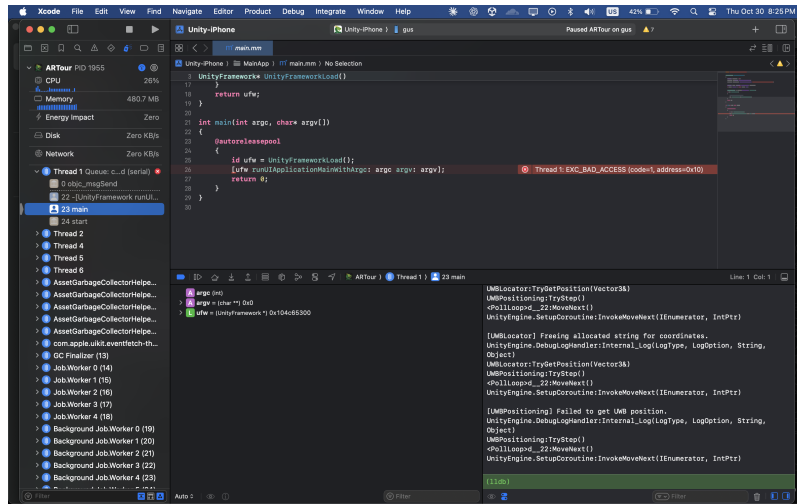


Figura 13: Log de comunicación entre la aplicación y el SDK de Estimote, donde se origina el error de calendarización.

En síntesis, la evaluación cualitativa del sistema permitió confirmar que el algoritmo de trilateración y el filtro de Kalman operan correctamente en condiciones estables, pero que la capa de comunicación —dependiente de software propietario cerrado— representa la principal limitante para la escalabilidad y confiabilidad del sistema en entornos complejos.

9.6. Conclusiones

- Se logró diseñar e implementar un algoritmo de trilateración bidimensional capaz de estimar la posición del usuario en interiores utilizando distancias medidas hacia múltiples sensores UWB. El modelo, basado en ecuaciones normales y extendido a multilateración, permitió reducir parcialmente la sensibilidad a errores individuales, produciendo posiciones coherentes y estables dentro del plano. No obstante, las pruebas experimentales revelaron la presencia de variabilidad y jitter en las mediciones, atribuibles al ruido inherente de los sensores y a las condiciones físicas del entorno. En consecuencia, el objetivo se considera cumplido, aunque con la necesidad de aplicar técnicas complementarias de filtrado para optimizar la estabilidad del sistema.
- Se logró integrar de manera efectiva el SDK nativo de Estimote con el entorno Unity a través del desarrollo de un plugin en Swift que actúa como puente entre ambas plataformas. La comunicación en tiempo real permitió transmitir datos de posición de los sensores hacia la aplicación de realidad aumentada con baja latencia y sin degradación perceptible del rendimiento. Este componente garantizó la interoperabilidad entre el sistema de localización nativo y el motor gráfico, sentando las bases para futuras extensiones multiplataforma. Por tanto, este objetivo se considera plenamente alcanzado.
- Se implementó exitosamente un filtro de Kalman que permitió fusionar los datos obtenidos por trilateración con las lecturas del acelerómetro del dispositivo, mejorando significativamente la suavidad y estabilidad de la trayectoria estimada. El filtro demostró ser eficaz para reducir el jitter y las oscilaciones espurias, especialmente en condiciones de reposo o movimiento uniforme. El proceso de ajuste de parámetros y validación experimental confirmó la utilidad del filtrado para alcanzar un equilibrio adecuado entre precisión y estabilidad visual, cumpliendo satisfactoriamente con el objetivo planteado.
- El desarrollo del plugin se acompañó de una documentación técnica exhaustiva que describe la arquitectura del sistema, la estructura de los módulos y el flujo de datos entre las capas nativas y Unity. Se aplicaron principios de diseño modular y buenas prácticas de programación orientada a objetos, facilitando la mantenibilidad y la futura extensión del sistema hacia dispositivos Android. Esta documentación garantiza la continuidad del megaproyecto, sirviendo como referencia técnica para futuras generaciones. Por ello, el objetivo se considera plenamente cumplido.
- La evaluación cualitativa del sistema de localización permitió confirmar la coherencia del movimiento estimado con la trayectoria real del usuario, validando la efectividad del algoritmo y del filtro de Kalman en entornos controlados. Sin embargo, se identificaron limitaciones impuestas por el SDK de Estimote, que restringieron la posibilidad de realizar mediciones consistentes en escenarios más amplios o con mayor número de sensores activos. A pesar de ello, los resultados obtenidos demuestran que el sistema cumple con su propósito de proporcionar una experiencia de navegación fluida y precisa dentro del entorno de realidad aumentada, considerándose este objetivo alcanzado parcialmente.

10. Recomendaciones

A partir de los resultados obtenidos, se plantean las siguientes recomendaciones con el propósito de fortalecer la investigación y orientar futuros desarrollos en el área:

- **Ampliar las pruebas de campo:** se recomienda evaluar el sistema en entornos más amplios y con obstáculos, con el fin de estudiar la degradación de la señal UWB y el desempeño del filtro de Kalman en condiciones no controladas.
- **Optimizar la calibración de los sensores:** pequeñas variaciones en la altura o el ángulo de los sensores pueden introducir errores sistemáticos en las mediciones. Una rutina de calibración automática previa a cada sesión podría mejorar la precisión global del sistema.
- **Explorar métodos de fusión sensorial avanzados:** se sugiere analizar la implementación de filtros de partículas o filtros de Kalman extendidos (EKF), que podrían ofrecer un mejor desempeño en presencia de modelos no lineales y distribuciones no gaussianas.
- **Completar y publicar la documentación técnica:** se recomienda consolidar la documentación del plugin, incluyendo diagramas de arquitectura, flujos de datos y guías de integración, de manera que futuras generaciones puedan continuar el desarrollo del megaproyecto sin dificultad.
- **Investigar la integración con visión por computadora:** como línea de trabajo futura, podría incorporarse información visual mediante técnicas de SLAM o reconocimiento de marcadores, combinando posicionamiento UWB y percepción visual para mejorar la precisión en zonas con interferencias.

11. Bibliografía

- Estimote, Inc. (n.d.). *Estimote Location Beacons: UWB and BLE*. Estimote Documentation. Recuperado de: <https://developer.estimote.com/proximity/ultra-wideband/>
- Estimote, Inc. (n.d.). *Estimote Proximity SDK*. Estimote Documentation. Recuperado de: <https://developer.estimote.com/proximity/ios-tutorial/>
- Unity Technologies. (n.d.). *AR Foundation overview*. Unity Documentation. Recuperado de: <https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@5.0/manual/index.html>
- Unity Technologies. (n.d.). *Unity Manual: ARKit and ARCore with AR Foundation*. Recuperado de: <https://docs.unity3d.com/Manual/com.unity.xr.arfoundation.html>
- Apple Inc. (n.d.). *Developing a Custom Unity Plugin for iOS*. Apple Developer / Unity Integration Guides. Recuperado de: <https://developer.apple.com/documentation/>
- Google. (n.d.). *Bluetooth Low Energy overview*. Android Developers. Recuperado de: <https://developer.android.com/guide/topics/connectivity/bluetooth-le>
- Apple Inc. (n.d.). *Core Bluetooth Overview*. Recuperado de: <https://developer.apple.com/documentation/corebluetooth>
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). *A formal basis for the heuristic determination of minimum cost paths*. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107. <https://doi.org/10.1109/TSSC.1968.300136>
- Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic Robotics*. MIT Press.
- Welch, G., & Bishop, G. (1995). *An Introduction to the Kalman Filter*. University of North Carolina at Chapel Hill. https://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf
- Ho, K. C., & Xu, Y. T. (2004). *An accurate algebraic solution for moving source location using TDOA and FDOA measurements*. IEEE Transactions on Signal Processing, 52(9), 2453–2463. <https://doi.org/10.1109/TSP.2004.832009>
- GitHub. (n.d.). *Unity Plugin Development for iOS and Android*. Ejemplos de integración. Recuperado de: <https://github.com/Unity-Technologies/UnityNativePlugin>

12. Anexos

12.1. Implementación del método de multilateración

```
1 private func multilateration(anchors: [Anchor]) -> Vector2D {
2     let (matrix, vector) = multInitializeMatrixVector(anchors: anchors)
3     let n = matrix.count / 3
4     let transposedMatrix = transpose(matrix: matrix, rows: n , cols: 3)
5
6     let lhs = matrixMultiplication(A: transposedMatrix, rowsOfA: 3,
7                                     B: matrix, colsOfB: 3, sharedDim: n)
8     let rhs = matrixMultiplication(A: transposedMatrix, rowsOfA: 3,
9                                     B: vector, colsOfB: 1, sharedDim: n)
10
11     let result = gaussJordan(A: lhs, n: 3, b: rhs)
12     return Vector2D(x: result[0], y: result[1])
13 }
```

Listing 1: Implementación del método de multilateración en Swift.

12.2. Implementación del método trilaterate 3D

```
1 private func tirlaterate_3D(anchors: [Anchor]) -> Vector2D{
2     let d1: Double = anchors[0].distance
3     let x1: Double = anchors[0].position.x
4     let y1: Double = anchors[0].position.y
5     let z1: Double = anchors[0].position.z
6
7     let d2: Double = anchors[1].distance
8     let x2: Double = anchors[1].position.x
9     let y2: Double = anchors[1].position.y
10    let z2: Double = anchors[1].position.z
11
12    let d3: Double = anchors[2].distance
13    let x3: Double = anchors[2].position.x
14    let y3: Double = anchors[2].position.y
15    let z3: Double = anchors[2].position.z
16
17    // First EQ
18    let A = 2 * (x1 - x2)
19    let B = 2 * (y1 - y2)
20    let C = 2 * (z1 - z2)
21
22    let D = pow(d2, 2) - pow(d1,2) - pow(x2,2) - pow(y2,2) - pow(z2,2) + pow(x1, 2) +
23        pow(y1, 2) + pow(z1, 2)
24
25    // Second EQ
26    let E = 2 * (x1 - x3)
27    let F = 2 * (y1 - y3)
28    let G = 2 * (z1 - z3)
29    let H = pow(d3, 2) - pow(d1, 2) - pow(x3, 2) - pow(y3, 2) - pow(z3, 2) + pow(x1, 2)
30        + pow(y1, 2) + pow(z1, 2)
```

```

30 // Aux EQ
31 let afeb = (A * F - E * B)
32
33 let alpha = (-B * G + C * F) / afeb
34 let beta = (-B * H + D * F) / afeb
35
36 let phi = (A * G - E * C) / afeb
37 let roh = (A * H - E * D) / afeb
38
39 // Coefficients
40 let first = pow(alpha, 2) + pow(phi, 2) + 1
41 let second = -(2 * alpha * beta) + (2 * x1 * alpha) - (2 * phi * roh) + (2 * y1 *
42     phi) - (2 * z1)
43 let third = pow(beta, 2) + pow(x1, 2) - (2 * x1 * beta) + pow(roh, 2) + pow(y1, 2) -
44     (2 * y1 * roh) + pow(z1, 2) - pow(d1, 2)
45
46 let discriminant = Double(pow(second, 2) - 4 * first * third)
47
48 var z: Double = 0.0
49
50 if discriminant > 0 {
51     if let value = solveQuadratic(a: first, b: second, c: third, discriminant:
52         discriminant){
53         z = value[0]
54     }
55 }
56
57 let x = Double(beta - alpha * z)
58 let y = Double(roh - phi * z)
59 return Vector2D(x: x, y: y)
60 }

```

Listing 2: Implementación del método trilaterate 3D

12.3. Implementación completa del algoritmo de trilateración en Swift

```

1 import Foundation
2
3 func do_trilateration(anchors: [String: Anchor]) -> Vector2D{
4     let anchorArray = Array(anchors.values)
5
6     if anchors.count == 3 {
7         // If exactly 3 anchors, do normal trilateration
8         let position = trilaterate_3D(anchors: anchorArray)
9         return position
10    } else {
11        // If more than 3, use multilateration
12        let position = multilateration(anchors: anchorArray)
13        return position
14    }
15 }
16
17 private func pickTopThreeAnchors(anchors: [String: Anchor]) -> [Anchor]{
18     if anchors.count == 3 {
19         return Array(anchors.values)

```

```

20     }
21
22     return anchors
23         .values
24         .compactMap { anchor in
25             anchor.timestamp != nil ? anchor : nil
26         }
27         .sorted { $0.timestamp! > $1.timestamp! }
28         .prefix(3)
29         .map { $0 }
30 }
31
32 private func multilateration(anchors: [Anchor]) -> Vector2D{
33     let (matrix, vector) = multInitializeMatrixVector(anchors: anchors)
34     let n = matrix.count/3
35
36     let transposedMatrix = transpose(matrix: matrix, rows: n , cols: 3)
37
38     let lhs = matrixMultiplication(A: transposedMatrix, rowsOfA: 3, B: matrix, colsOfB:
39         3, sharedDim: n)
40
41     let rhs = matrixMultiplication(A: transposedMatrix, rowsOfA: 3, B: vector, colsOfB:
42         1, sharedDim: n)
43
44     let result = gaussJordan(A: lhs, n: 3, b: rhs)
45     print("Multilat result: \n(result)")
46
47     return Vector2D(x:result[0], y:result[1])
48 }
49
50 private func multInitializeMatrixVector(anchors: [Anchor])-> (matrix: [Double], vector:
51     [Double]){
52
53     var anchorsCopy = anchors
54     guard let pivot = anchorsCopy.popLast() else {
55         fatalError("pivotAnchor is nil")
56     }
57
58     var matrix: [Double] = []
59     var vector: [Double] = []
60
61     for anchor in anchors{
62
63         matrix.append(contentsOf: [
64             2 * (pivot.position.x - anchor.position.x),
65             2 * (pivot.position.y - anchor.position.y),
66             2 * (pivot.position.z - anchor.position.z)
67         ])
68
69         let dA2 = pow(anchor.distance, 2)
70         let dP2 = pow(pivot.distance, 2)
71
72         let ax2 = pow(anchor.position.x, 2)
73         let ay2 = pow(anchor.position.y, 2)
74         let az2 = pow(anchor.position.z, 2)

```

```

73     let px2 = pow(pivot.position.x, 2)
74     let py2 = pow(pivot.position.y, 2)
75     let pz2 = pow(pivot.position.z, 2)
76
77     let value = dA2 - dP2 - ax2 - ay2 - az2 + px2 + py2 + pz2
78
79     vector.append(value)
80 }
81
82 return (matrix, vector)
83 }
84
85 private func tirlaterate_3D(anchors: [Anchor]) -> Vector2D{
86     let d1: Double = anchors[0].distance
87     let x1: Double = anchors[0].position.x
88     let y1: Double = anchors[0].position.y
89     let z1: Double = anchors[0].position.z
90
91     let d2: Double = anchors[1].distance
92     let x2: Double = anchors[1].position.x
93     let y2: Double = anchors[1].position.y
94     let z2: Double = anchors[1].position.z
95
96     let d3: Double = anchors[2].distance
97     let x3: Double = anchors[2].position.x
98     let y3: Double = anchors[2].position.y
99     let z3: Double = anchors[2].position.z
100
101     // First EQ
102     let A = 2 * (x1 - x2)
103     let B = 2 * (y1 - y2)
104     let C = 2 * (z1 - z2)
105
106     let D = pow(d2, 2) - pow(d1,2) - pow(x2,2) - pow(y2,2) - pow(z2,2) + pow(x1, 2) +
        pow(y1, 2) + pow(z1, 2)
107
108     // Second EQ
109     let E = 2 * (x1 - x3)
110     let F = 2 * (y1 - y3)
111     let G = 2 * (z1 - z3)
112     let H = pow(d3, 2) - pow(d1, 2) - pow(x3, 2) - pow(y3, 2) - pow(z3, 2) + pow(x1, 2)
        + pow(y1, 2) + pow(z1, 2)
113
114     // Aux EQ
115     let afeb = (A * F - E * B)
116
117     let alpha = (-B * G + C * F) / afeb
118     let beta = (-B * H + D * F) / afeb
119
120     let phi = (A * G - E * C) / afeb
121     let roh = (A * H - E * D) / afeb
122
123     // Coefficients
124     let first = pow(alpha, 2) + pow(phi, 2) + 1
125     let second = -(2 * alpha * beta) + (2 * x1 * alpha) - (2 * phi * roh) + (2 * y1 *
        phi) - (2 * z1)

```

```

126     let third = pow(beta, 2) + pow(x1, 2) - (2 * x1 * beta) + pow(roh, 2) + pow(y1, 2) -
        (2 * y1 * roh) + pow(z1, 2) - pow(d1, 2)
127
128     let discriminant = Double(pow(second, 2) - 4 * first * third)
129
130     var z: Double = 0.0
131
132     if discriminant > 0 {
133         if let value = solveQuadratic(a: first, b: second, c: third, discriminant:
            discriminant){
134             z = value[0]
135         }
136     }
137
138     let x = Double(beta - alpha * z)
139     let y = Double(roh - phi * z)
140     return Vector2D(x: x, y: y)
141 }

```

Listing 3: Implementación del algoritmo de multilateración en Swift.

12.4. Glosario

UWB (Ultra-Wideband): Tecnología de comunicación inalámbrica de corto alcance que utiliza pulsos de radio de amplio espectro para medir distancias con alta precisión. Es fundamental en sistemas de localización en interiores debido a su baja latencia y alta resolución temporal.

Trilateración: Método geométrico que determina la posición de un punto desconocido mediante la intersección de tres circunferencias (en 2D) o esferas (en 3D), cada una centrada en un sensor con radio igual a la distancia medida hacia el punto objetivo.

Multilateración: Extensión de la trilateración que emplea más de tres sensores para resolver un sistema sobredeterminado. Permite mejorar la precisión y robustez del cálculo de posición mediante técnicas de mínimos cuadrados.

Ecuación normal: Expresión matricial $A^T A x = A^T b$ utilizada para resolver sistemas lineales sobredeterminados. En el contexto del proyecto, se aplica para estimar la posición del usuario cuando existen más mediciones que incógnitas.

Filtro de Kalman: Algoritmo recursivo de estimación que combina mediciones ruidosas con un modelo dinámico del sistema para obtener una estimación óptima del estado. En esta investigación se utilizó para suavizar las posiciones estimadas por UWB y combinar los datos del acelerómetro.

Jitter: Variación errática y de alta frecuencia en las mediciones de posición o señal. En este trabajo, el término se refiere a la inestabilidad observada en la trilateración antes de aplicar filtrado.

Fusión sensorial: Técnica que combina múltiples fuentes de datos (por ejemplo, UWB y acelerómetro) para mejorar la precisión, estabilidad y confiabilidad de las estimaciones. Es la base del filtro de Kalman implementado.

Acelerómetro: Sensor capaz de medir la aceleración del dispositivo en los tres ejes cartesianos. En este proyecto se utilizó como fuente auxiliar para detectar movimiento y estabilizar el cálculo de posición.

CoreMotion: Framework de Apple que permite acceder a los datos de movimiento y orientación del dispositivo, incluyendo acelerómetro, giroscopio y magnetómetro.

SDK (Software Development Kit): Conjunto de herramientas y bibliotecas que facilitan la integración de funcionalidades específicas en una aplicación. En este trabajo, se utilizó el SDK de Estimote para interactuar con los sensores UWB.

Plugin nativo: Componente de software que permite integrar código escrito en un lenguaje nativo (como Swift u Objective-C) dentro de un motor multiplataforma como Unity, facilitando la comunicación entre ambas capas.

Unity: Motor de desarrollo multiplataforma utilizado para crear aplicaciones 3D e interactivas. En esta investigación se empleó como entorno de visualización y renderizado del sistema de posicionamiento.

Swift: Lenguaje de programación desarrollado por Apple para el desarrollo de aplicaciones en iOS. Fue utilizado para implementar el algoritmo de trilateración, multilateración y filtrado de Kalman.

Objective-C: Lenguaje de programación compatible con Swift que se utiliza para exponer funciones del código nativo hacia otros entornos, como Unity, mediante una interfaz de comunicación C-friendly.

Delegate (patrón de diseño): Patrón arquitectónico que permite delegar tareas específicas a diferentes objetos para reducir el acoplamiento entre módulos. En este proyecto se utilizó para separar la lógica de sensores UWB, acelerómetro y procesamiento de posición.

Realidad aumentada (RA): Tecnología que superpone información digital sobre el entorno físico percibido a través de una cámara o visor. El sistema de posicionamiento desarrollado busca ser la base de una aplicación de RA para interiores.

Varianza: Medida estadística de la dispersión de un conjunto de datos respecto a su media. En el contexto del proyecto, se utilizó para cuantificar el ruido presente en las mediciones UWB y del acelerómetro.

Modelo de estado: Representación matemática del sistema dinámico que describe cómo evoluciona el estado (posición, velocidad, etc.) con el tiempo. Es un componente esencial en el funcionamiento del filtro de Kalman.

Ruido de medición: Componente aleatorio no deseado presente en las observaciones de un sensor. Su reducción o compensación es uno de los objetivos principales del filtrado aplicado en el proyecto.

Sistema sobredeterminado: Sistema de ecuaciones con más ecuaciones que incógnitas, común en problemas de estimación con redundancia de sensores. Se resuelve minimizando el error cuadrático medio mediante ecuaciones normales.

Hiperparámetros: Parámetros configurables del filtro de Kalman (como las matrices de covarianza del proceso y de medición) que determinan su capacidad de adaptación al ruido y la dinámica del sistema.